LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Muhamad Radja Nur Akbar (2509106012)

Kelas (A1'25)

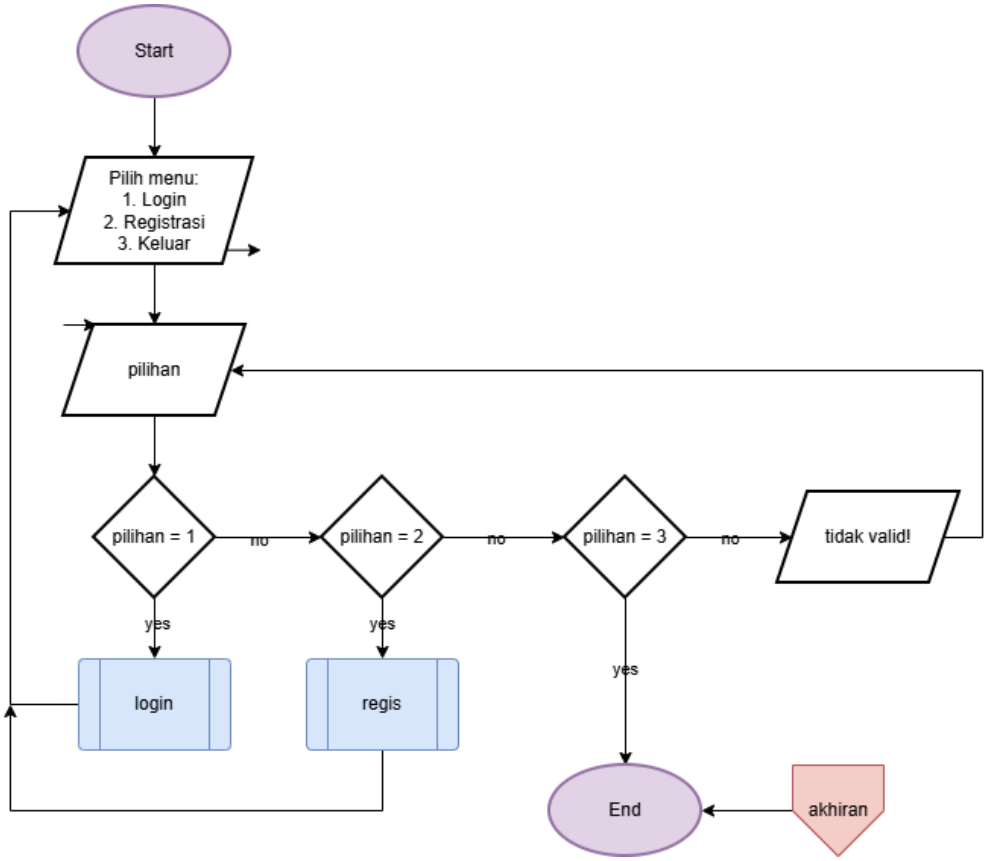
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

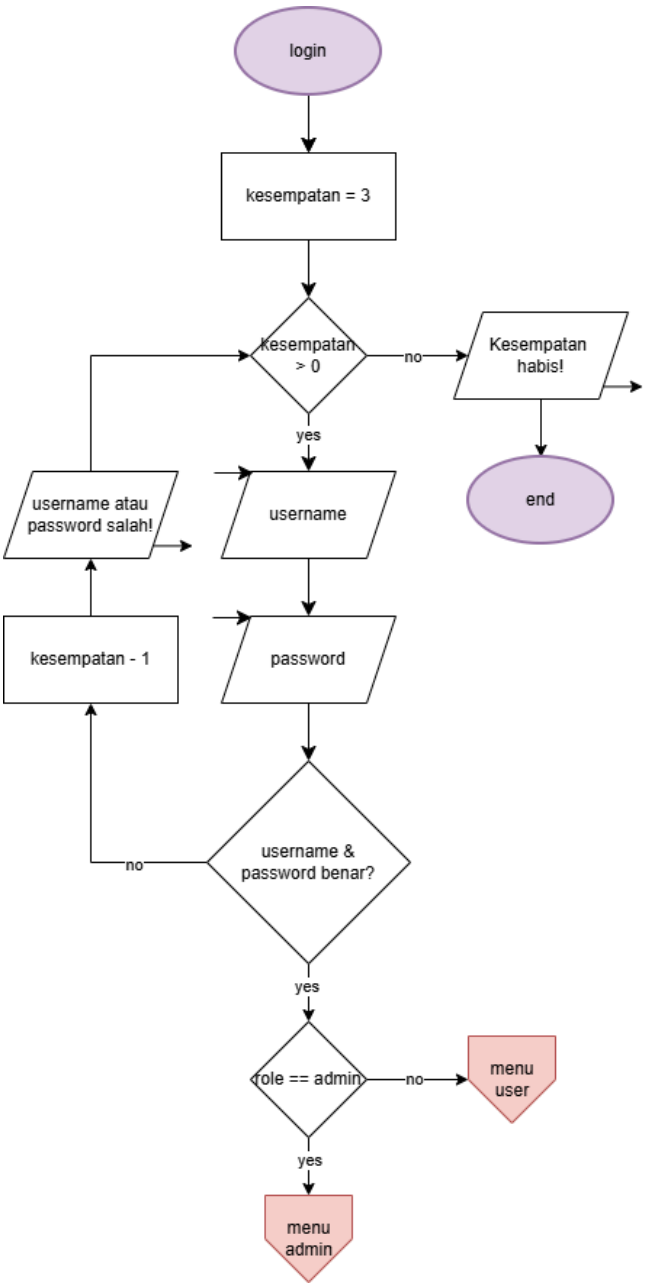
2026

1. Flowchart



Gambar 1.1 Menu Autentikasi

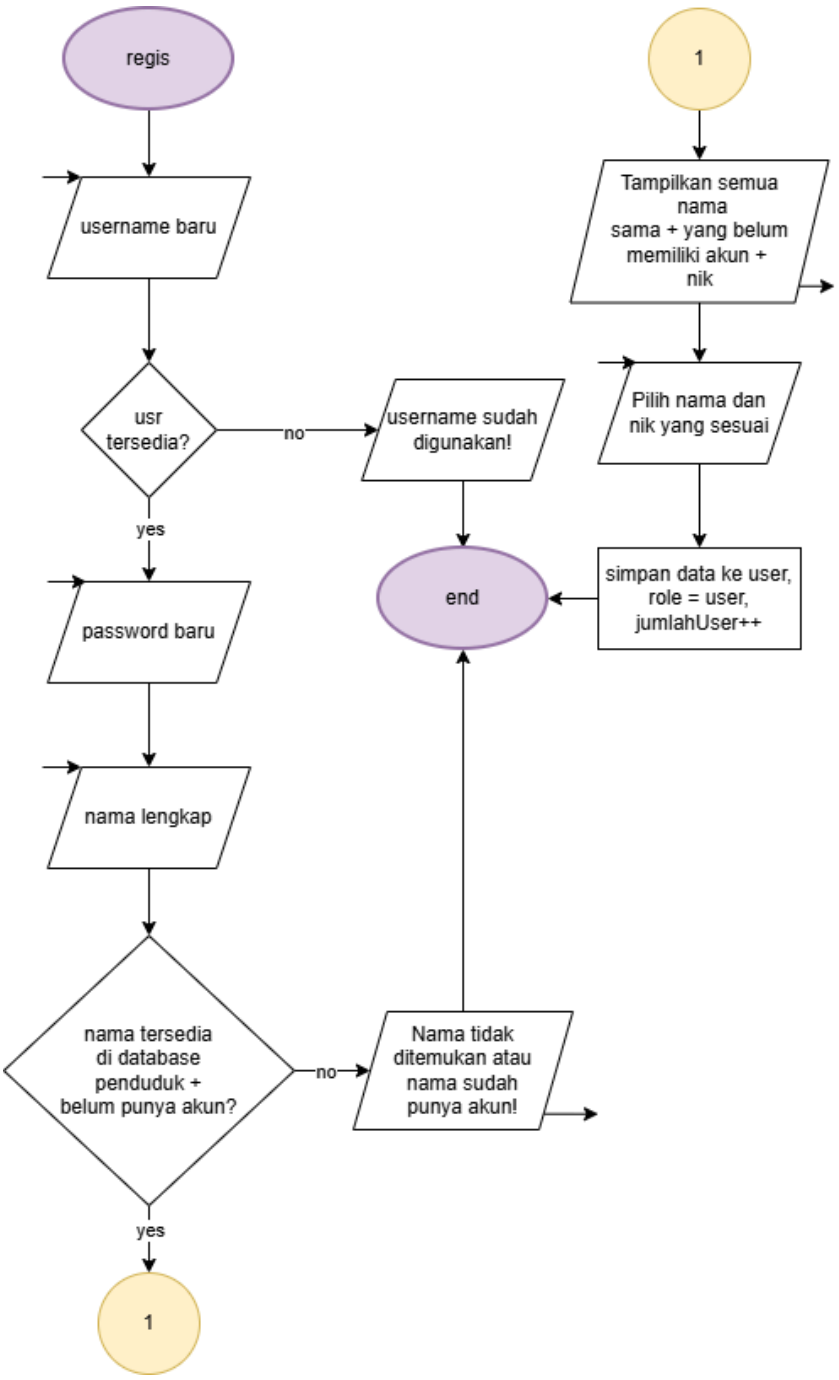
Tampilan awal dari program ini adalah menu autentikasi. Pengguna bisa memilih *login*, registrasi, atau keluar program. Kemudian buka fungsi yang sesuai.



Gambar 1.2 Fungsi Login

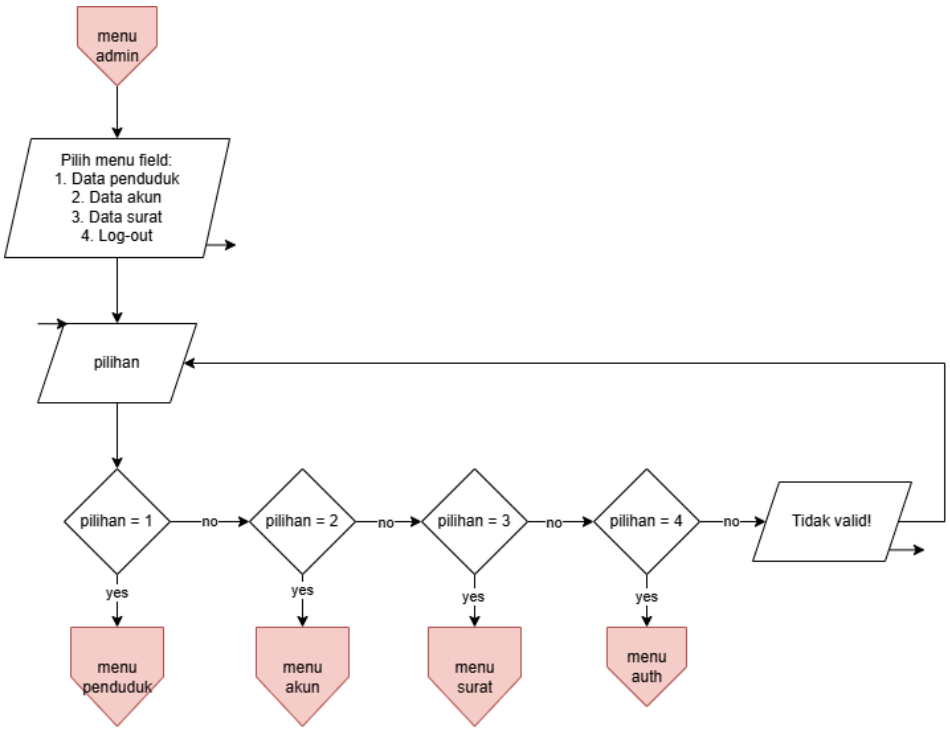
Dalam fungsi *login*, pengguna punya 3 kesempatan untuk *login*, setiap kesalahan akan mengurangi kesempatan sebanyak 1. Jika habis program berhenti.

Ketika *login* sudah tepat, *role* dari akun tersebut dicek untuk membawa pengguna ke menu yang sesuai. Ada dua, menu admin dan menu *user.*



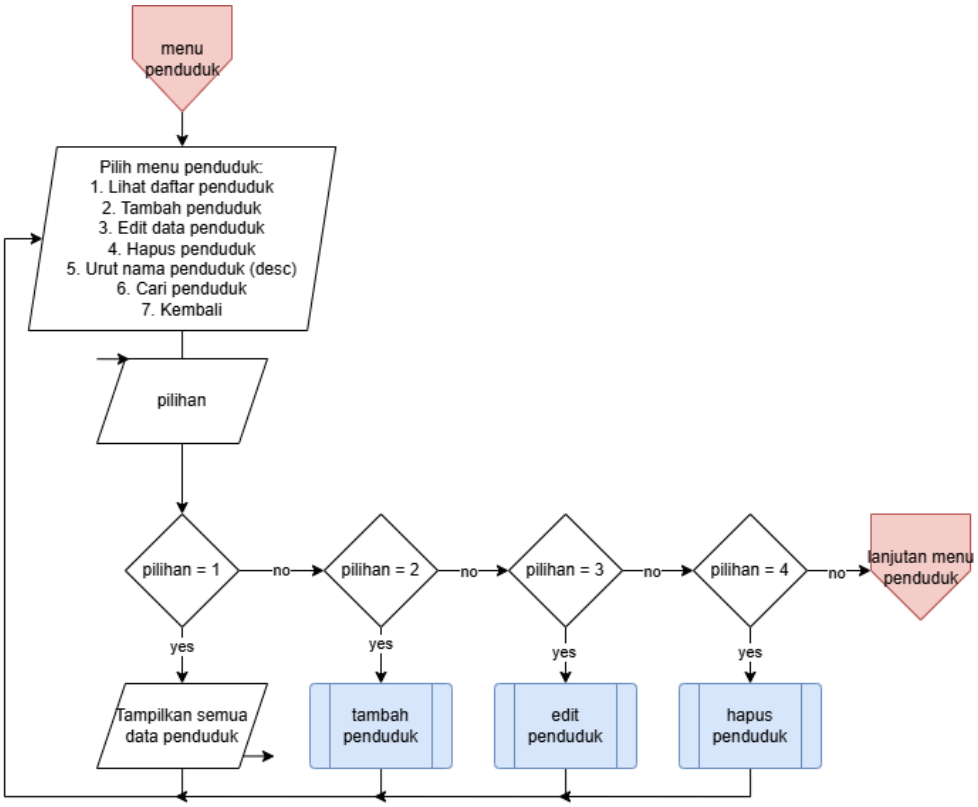
Gambar 1.3 Fungsi Registrasi

Dalam fungsi ini, masukkan *username* baru yang akan ditolak jika *username* sudah digunakan. Masukkan *password* dan nama lengkap. **Catatan:** pengguna hanya bisa registrasi jika mereka terdaftar di *database* penduduk. Jika terdapat nama yang sama dari *database* maka semua nama tersebut ditunjukkan beserta NIK supaya pengguna bisa memilih data yang tepat. Terakhir simpan *username* dan *password* dengan *role* bawaan adalah *user.*

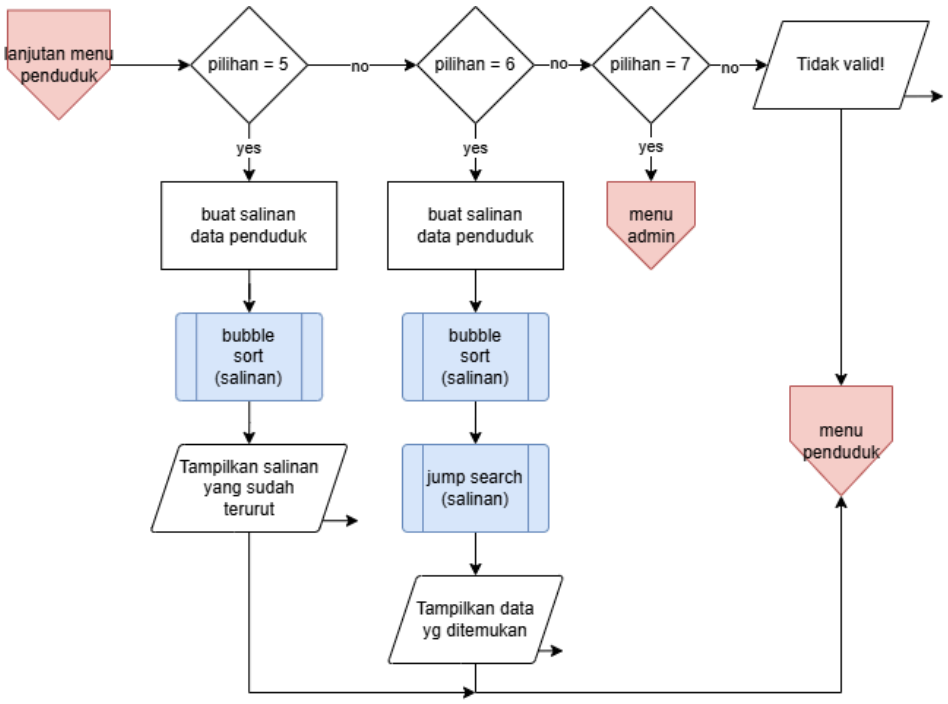


Gambar 1.4 Menu Admin

Admin dapat melakukan 3 jenis CRUD dalam kepengurusan balai kota, yaitu mengurus data penduduk, data akun, dan data surat. Admin hanya perlu memilih salah satu menu untuk melanjutkan.

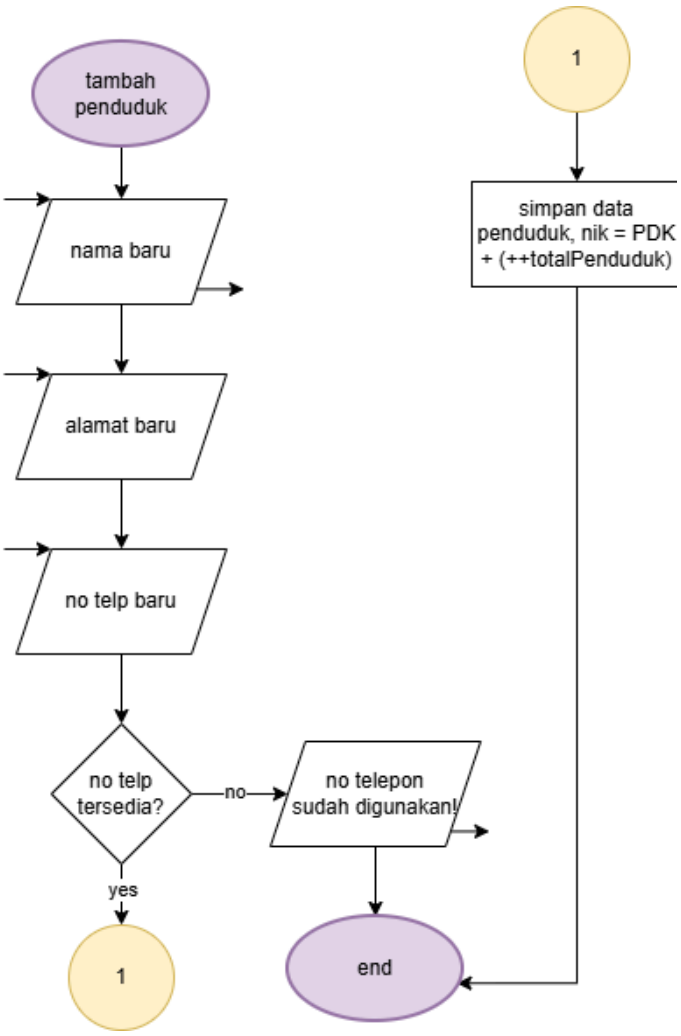


Gambar 1.5 Menu Penduduk



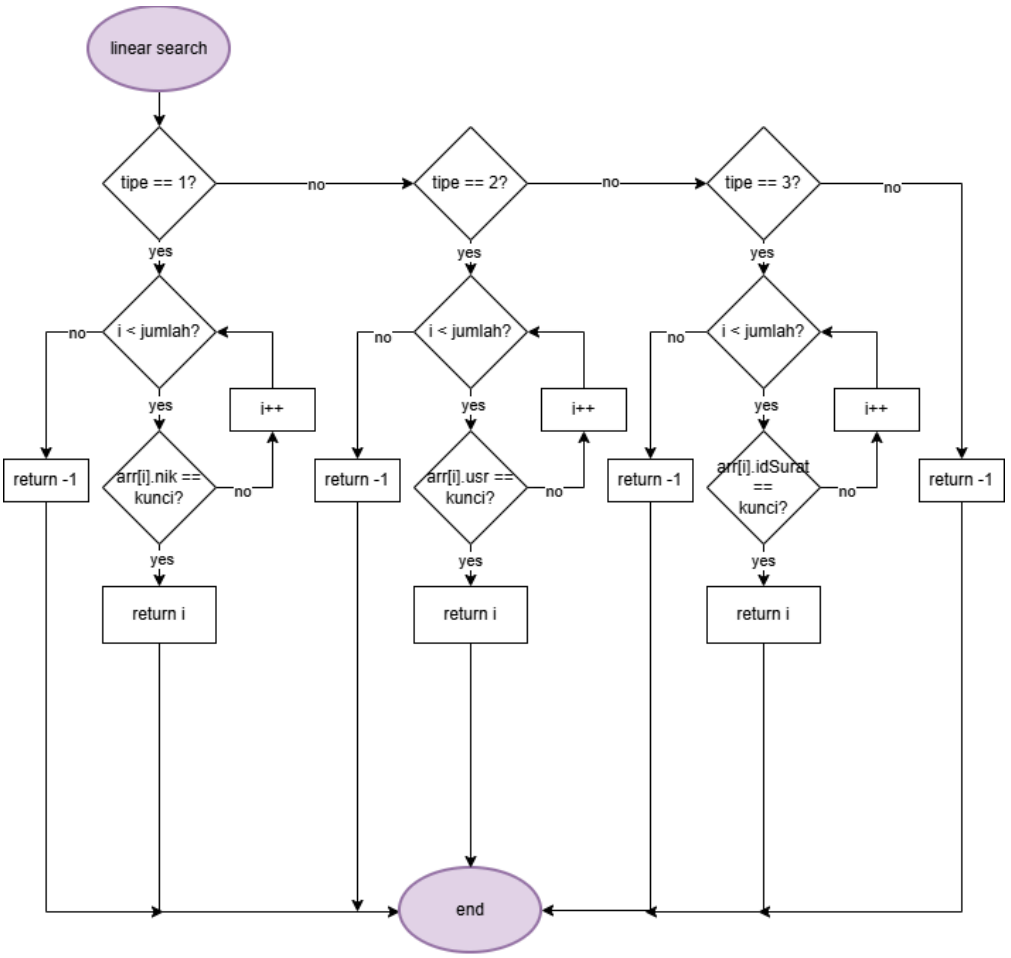
Gambar 1.6 Lanjutan Menu Penduduk

Dalam menu penduduk admin punya 6 pilihan dalam mengatur data penduduk. Melihat daftar penduduk, menambah penduduk, mengedit data penduduk, menghapus penduduk, mengurutkan data penduduk berdasarkan nama penduduk dan mencari penduduk berdasarkan nama.



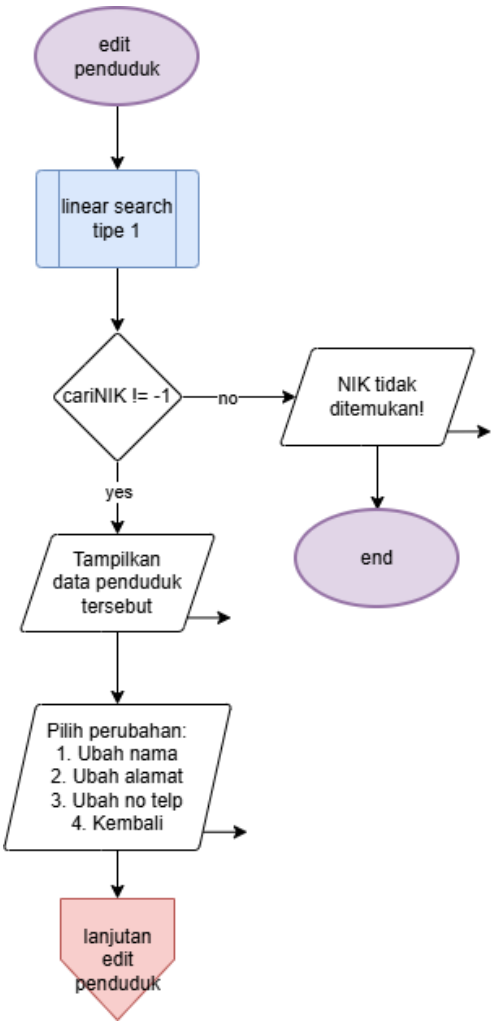
Gambar 1.7 Fungsi Tambah Penduduk

Di fungsi ini, penduduk memiliki 4 *subfield* yaitu NIK sebagai variabel identitas utama, nama, alamat, dan nomor telepon. Dalam menu di atas nomor telepon juga merupakan variabel unik yang tidak bisa sama satu sama lain. Ketika semua *subfield* sudah diisi variabel NIK akan otomatis terisi dengan menggunakan 3 huruf "PDK" ditambah total penduduk saat ini.

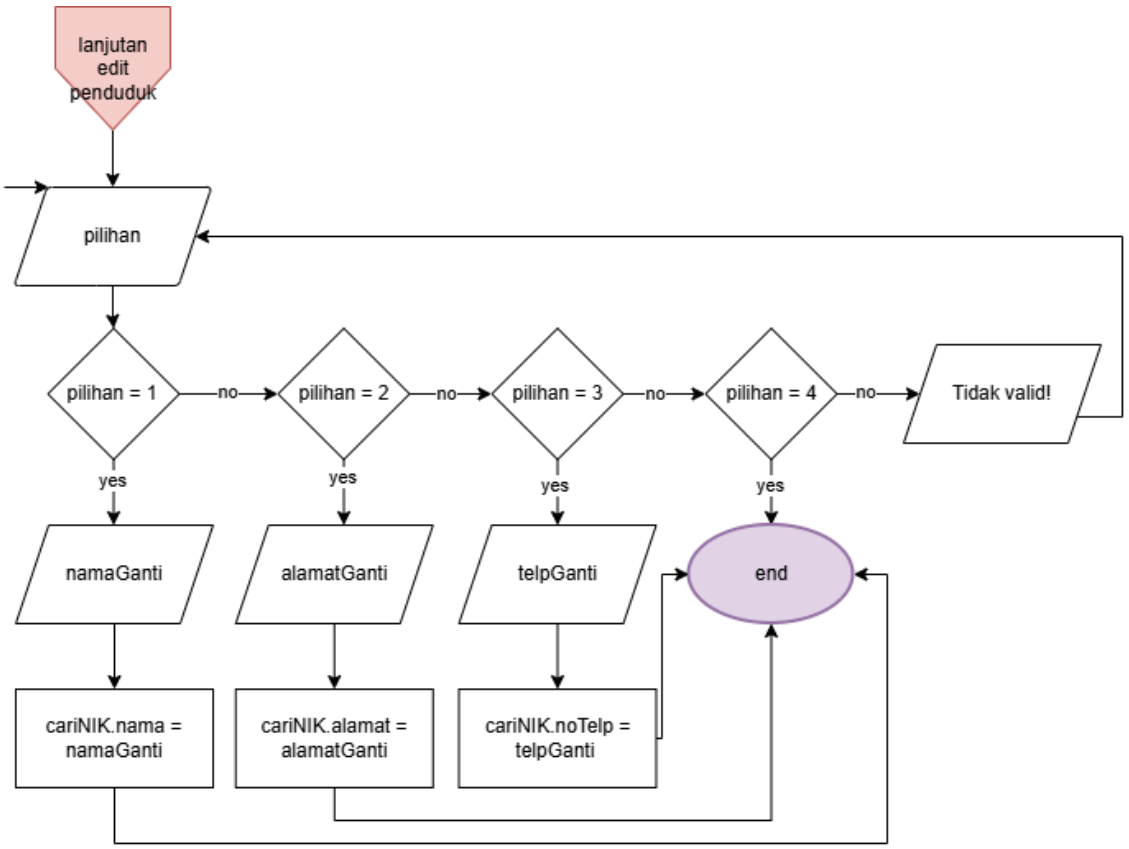


Gambar 1.8 Linear Search

Linear search berfungsi untuk mencari data dari ketiga jenis data ketika ada data yang ingin diedit atau dihapus. Metode ini menggunakan perbandingan satu per satu dengan kata kunci hingga ketemu data yang tepat. Baik terurut maupun tidak, *linear search* bergantung pada jumlah data. Efektif jika sedikit dan jika data kompleks namun buruk jika data sangat banyak. Dari *flowchart* di atas percabangan yang benar akan berjalan tergantung dari tipenya, jadi dari hanya satu fungsi ketiga jenis data dapat dijalankan.

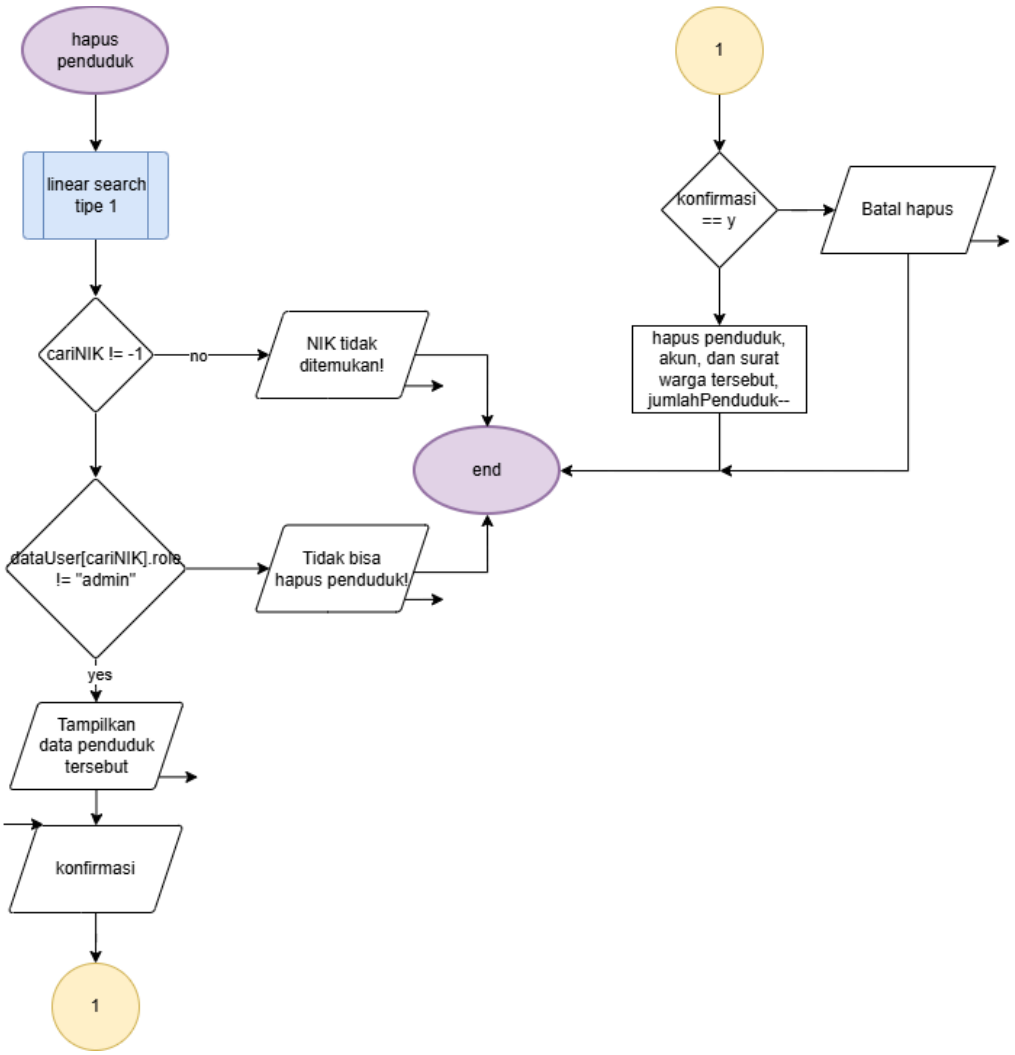


Di fungsi edit data penduduk, data penduduk yang dicari akan ditampilkan lalu admin bisa memilih apa yang akan diubah.



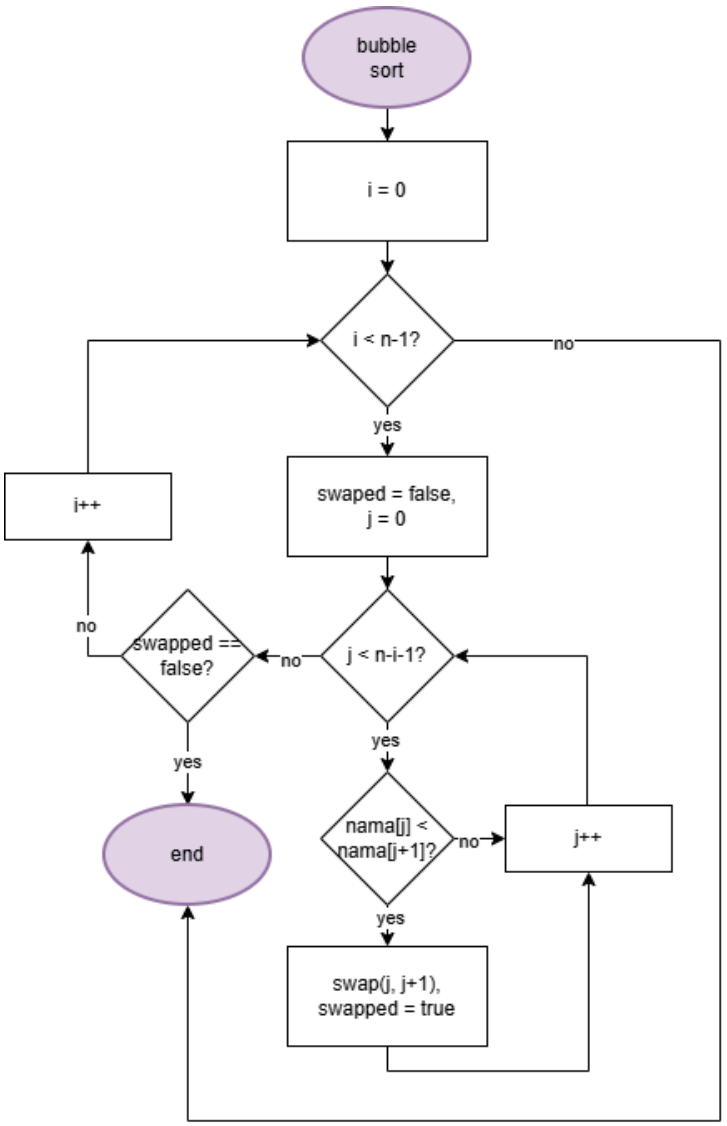
Gambar 1.10 Mengedit Data Penduduk

Admin dapat mengubah 3 *subfield* data penduduk. Prosesnya sama ketika menambah penduduk termasuk nomor telepon unik. Setelah mengganti data lama akan ditimpa dengan data baru.



Gambar 1.11 Fungsi Hapus Penduduk

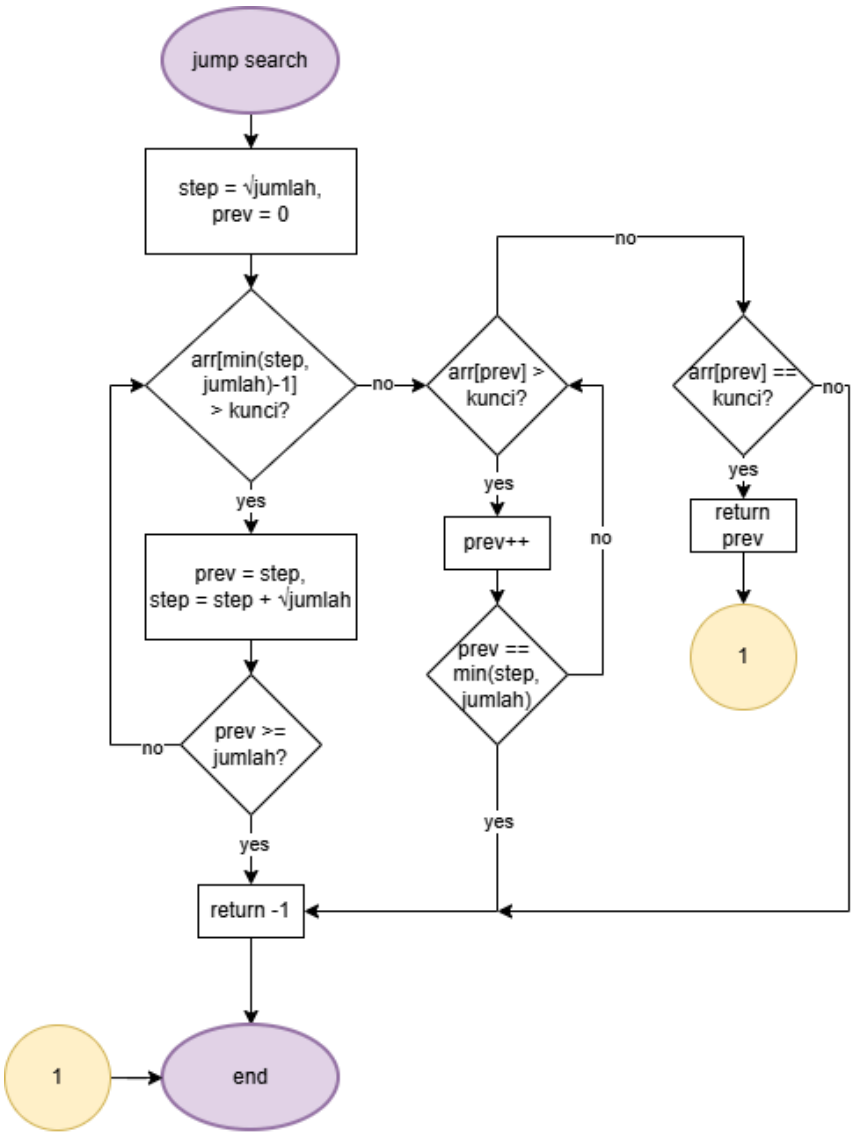
Seperti saat akan mengedit data penduduk, admin juga harus memasukkan NIK yang diinginkan untuk memulai proses penghapusan. Adapun syarat tambahan yaitu admin tidak dapat menghapus akun sesama admin. Admin diminta konfirmasi sebelum betul-betul menghapus data penduduk. Ketika data suatu penduduk dihapus maka akun dan semua surat dari penduduk tersebut juga akan dihapus.



Gambar 1.12 Bubble Sort

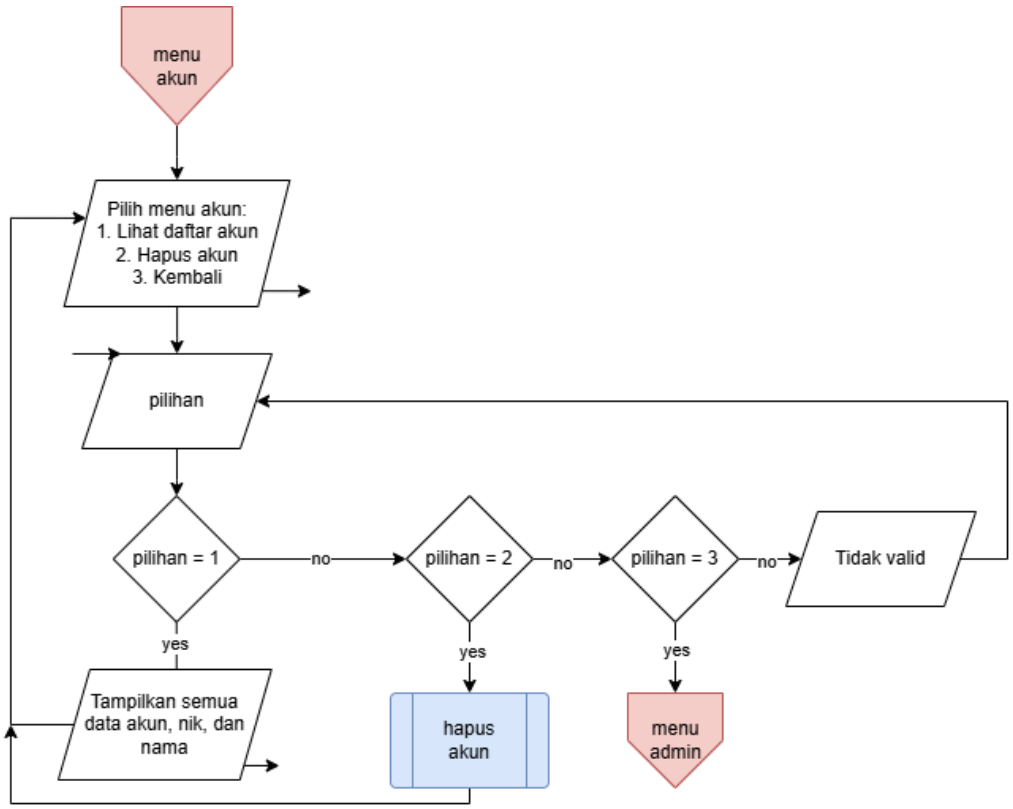
Metode *sorting* yang digunakan untuk mengurutkan data penduduk berdasarkan nama adalah *bubble sort*. Untuk menjaga keaslian urutan data, dibuatlah salinan data asli sebagai wadah *sorting*.

Algoritma *sorting* ini memiliki konsep pertukaran elemen. *Bubble* melihat elemen awal dengan elemen setelahnya lalu melakukan pertukaran apabila elemen setelah lebih besar khusus kasus ini yaitu *descending* (dari besar ke kecil).



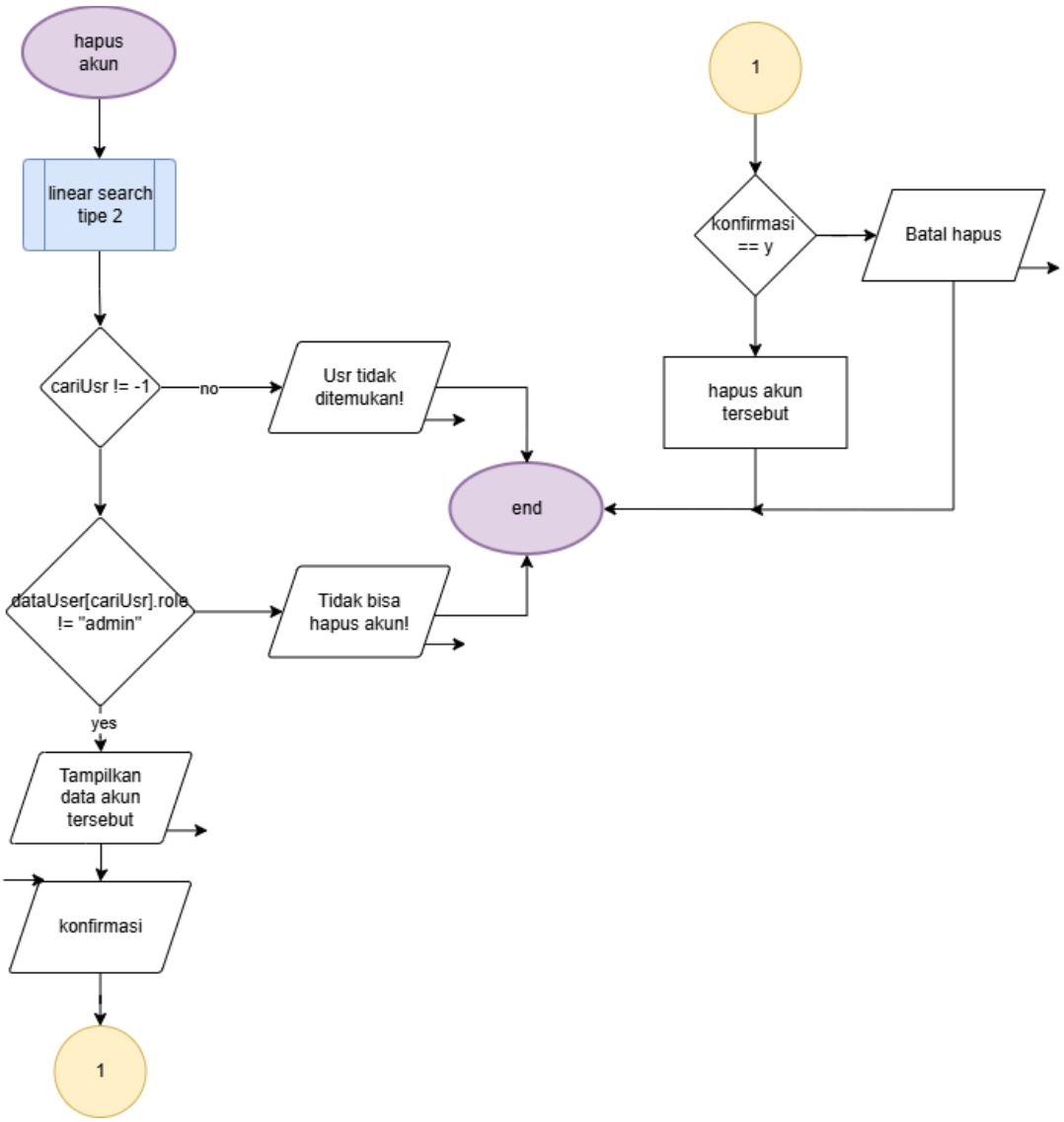
Gambar 1.13 Jump Search

Jump search merupakan metode *searching* yang bekerja dengan cara melompati beberapa elemen untuk menemukan target. Pencarian akan melompati beberapa elemen terlebih dahulu, lalu ketika sudah mendekati pencarian akan menggunakan *linear search*. Dalam kasus di atas *jump search* digunakan dalam pencarian penduduk berdasarkan nama, karena data telah diurutkan oleh *bubble sort* secara *descending*, maka perbandingan menggunakan operator lebih dari.



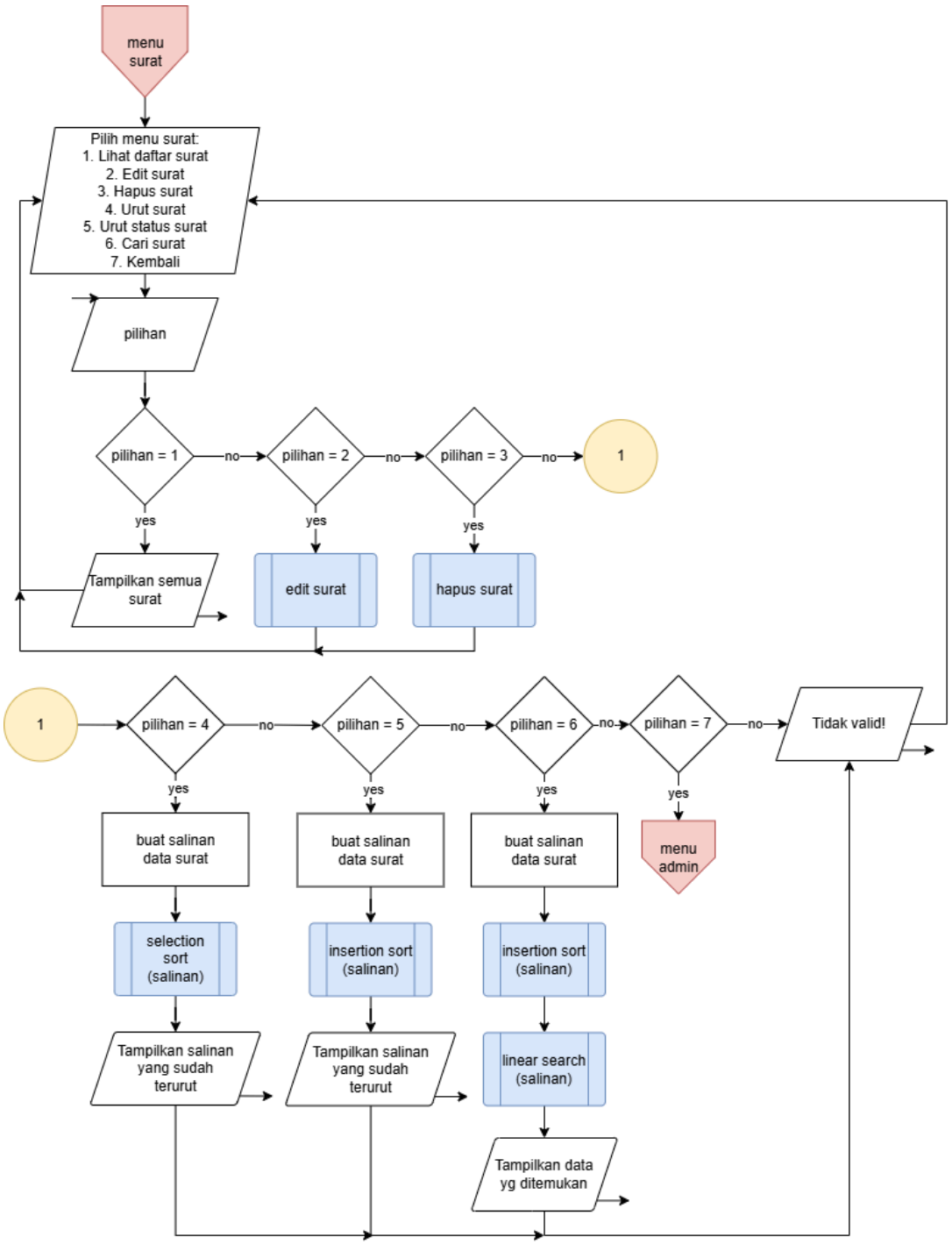
Gambar 1.14 Menu Akun

Setelah selesai di menu *field* penduduk, admin juga bisa memanipulasi akun-akun yang terdaftar. Memilih pilihan 1 akan menampilkan semua akun yang terdaftar beserta *role*, NIK, dan nama.



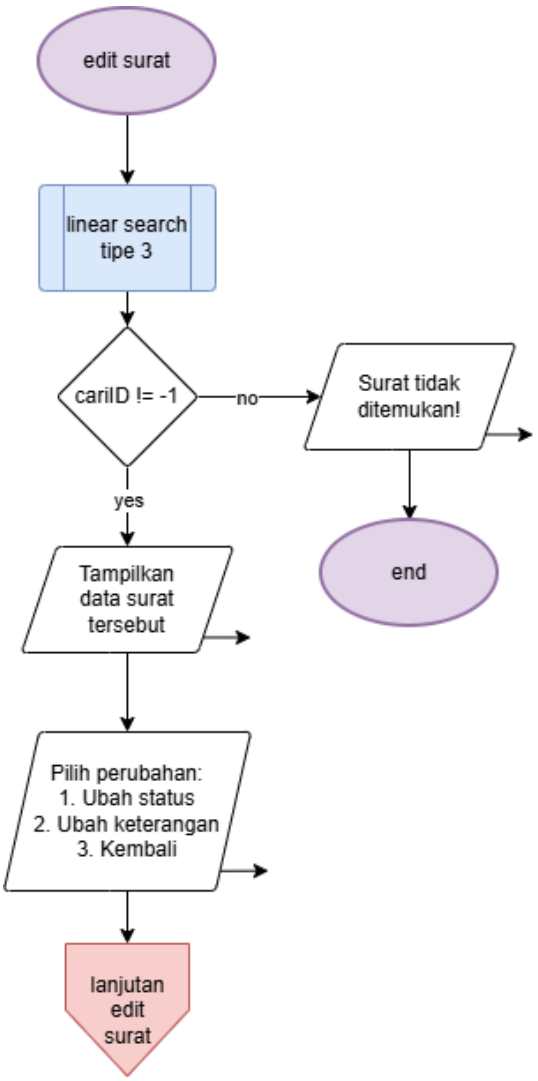
Gambar 1.15 Fungsi Hapus Akun

Fungsi ini berjalan sama seperti hapus penduduk, akun tidak bisa dihapus jika memiliki *role* admin. Setelah konfirmasi barulah akun tersebut dihapus.



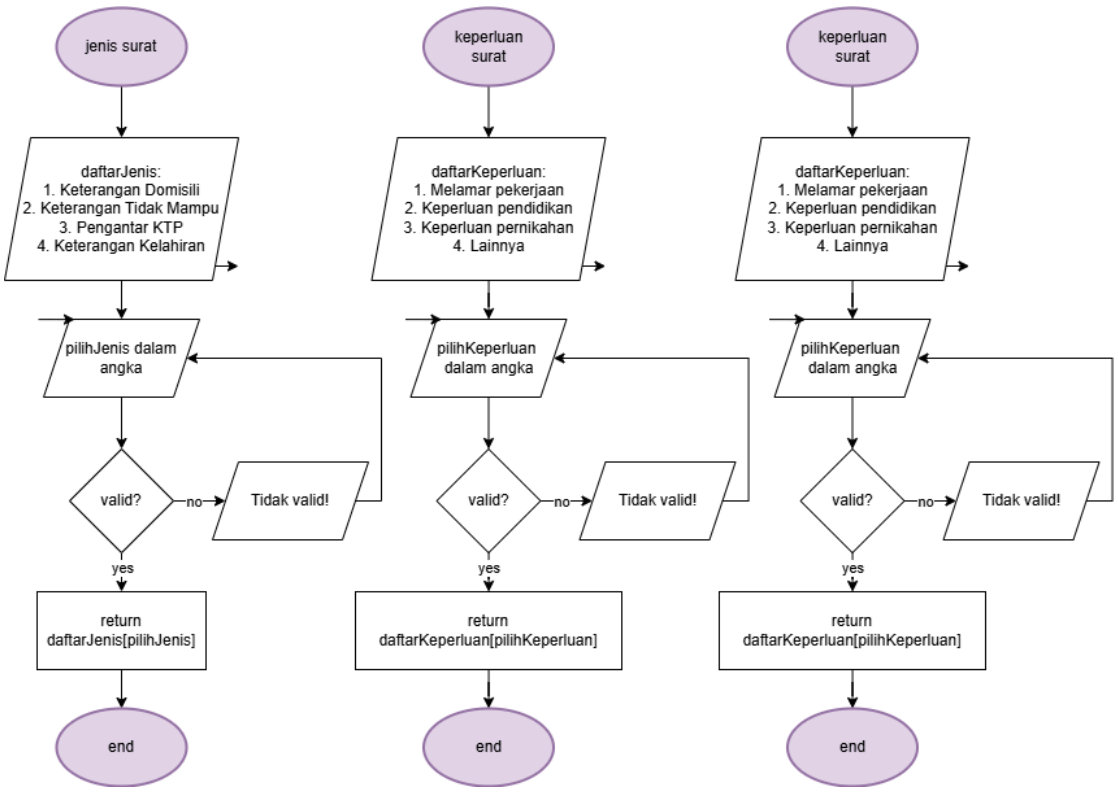
Gambar 1.16 Menu Surat Admin

Pada pilihan ketiga admin dapat mengubah data pada surat-surat ajuan. Data surat punya 6 subfield, yaitu ID surat, NIK pemohon, jenis surat, keperluan, status, keterangan. Admin dapat melihat daftar surat, mengedit surat, menghapus surat, mengurutkan surat berdasarkan ID surat dan status, serta mencari surat dari ID surat.



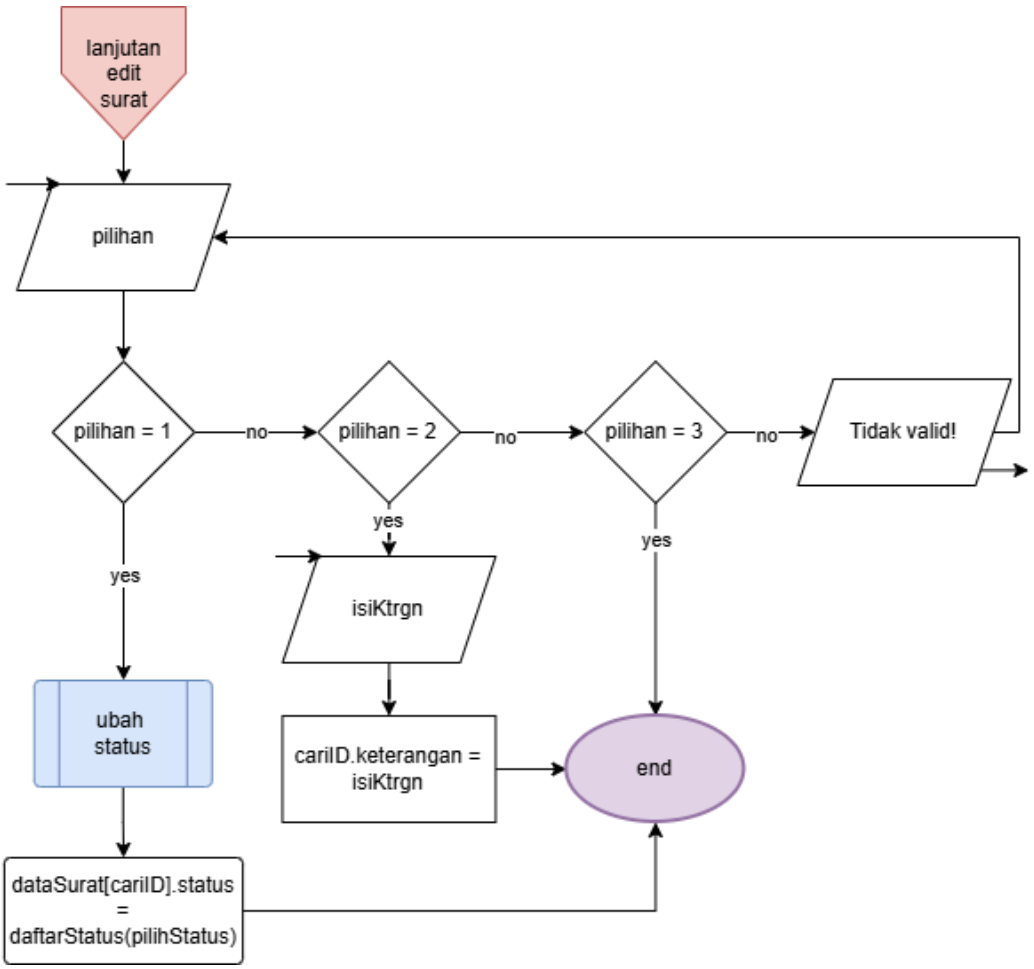
Gambar 1.17 Edit Surat Versi Admin

Panggil fungsi cari surat seperti pada *field* sebelumnya. Lalu pilih perubahan yang diinginkan.



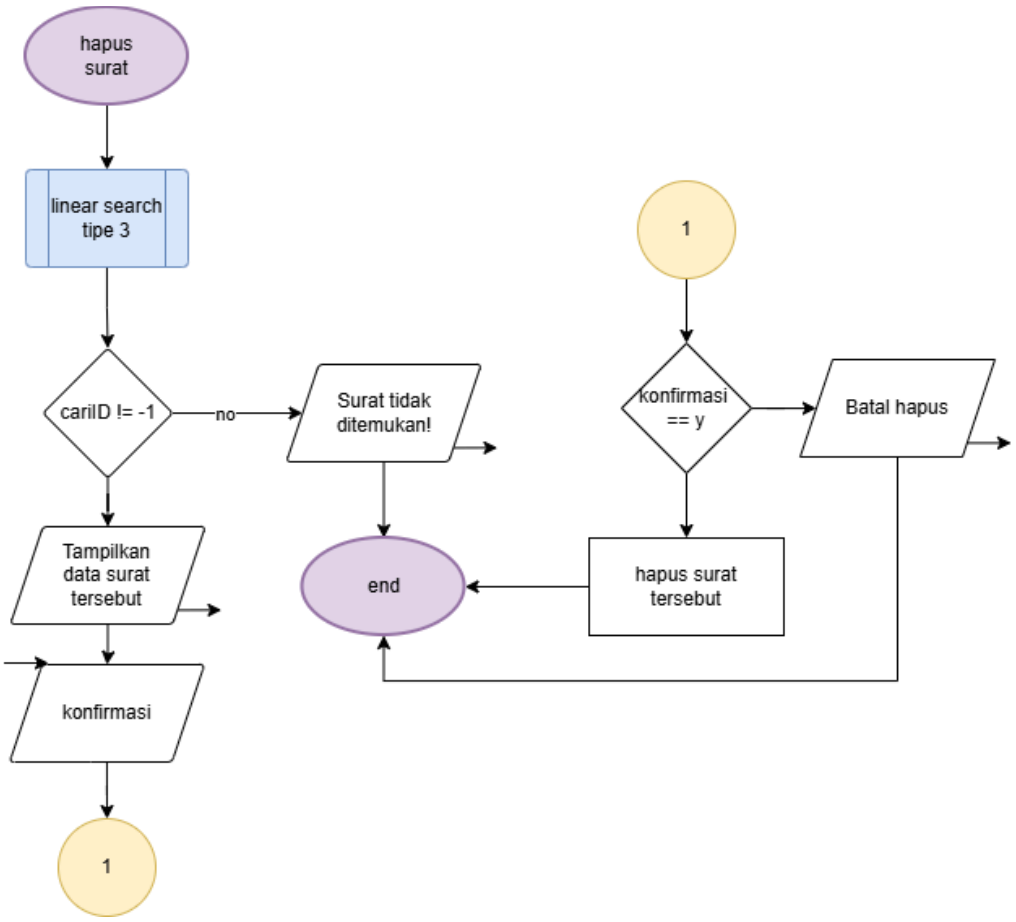
Gambar 1.18 Fungsi Ubah Subfield Surat

Jenis, keperluan, dan status merupakan *subfield* pada surat yang menggunakan fungsi sendiri. Digunakan dalam pengeditan dan pembuatan surat, masing-masing berjalan dengan cara yang sama. Setiap *subfield* memiliki daftar pilihan, kemudian disimpan dalam *integer*.



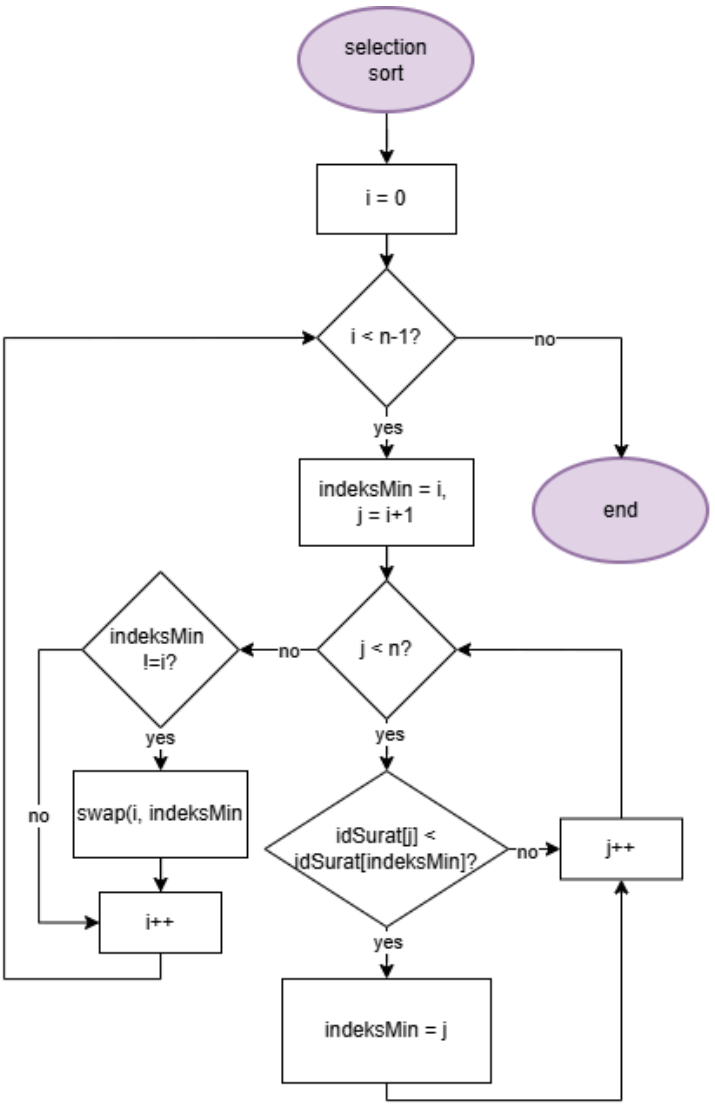
Gambar 1.19 Lanjutan Edit Surat Versi Admin

Admin tidak dapat mengubah data surat sepenuhnya, hanya di bagian status dan keterangan. Status menunjukkan perubahan surat pada waktu ke waktu, sementara keterangan menunjukkan deskripsi surat yang lebih detail kepada pemohon.



Gambar 1.20 Hapus Surat Versi Admin

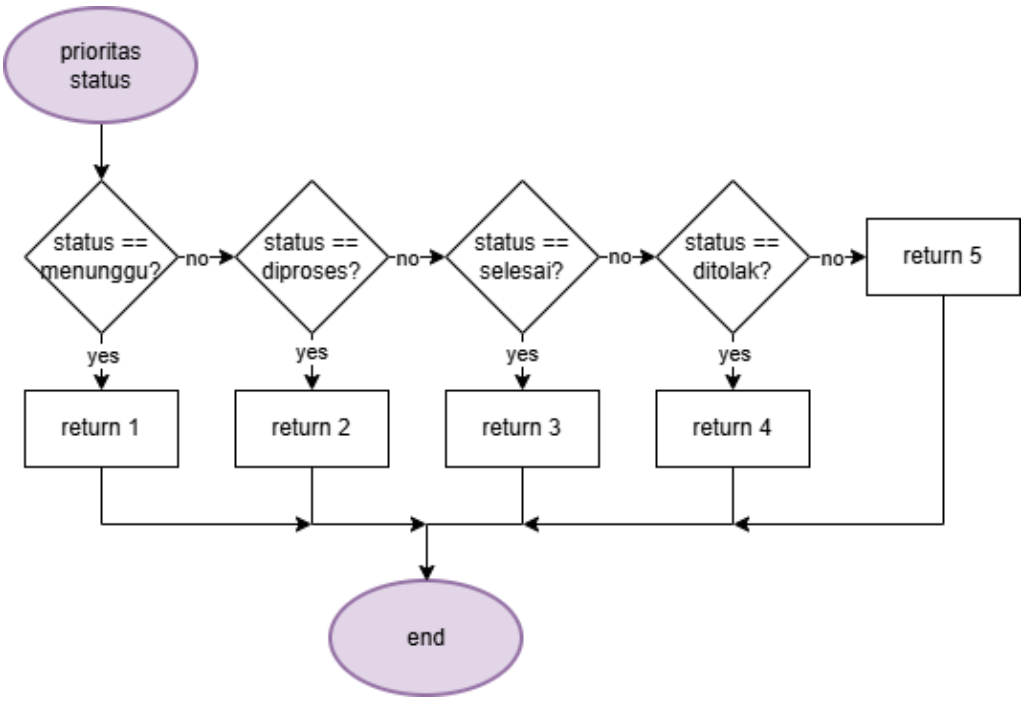
Prosesnya sama dengan hapus penduduk dan hapus akun. Masukkan ID surat pada dengan fungsi, meminta konfirmasi, dan hapus surat tersebut.



Gambar 1.21 Selection Sort

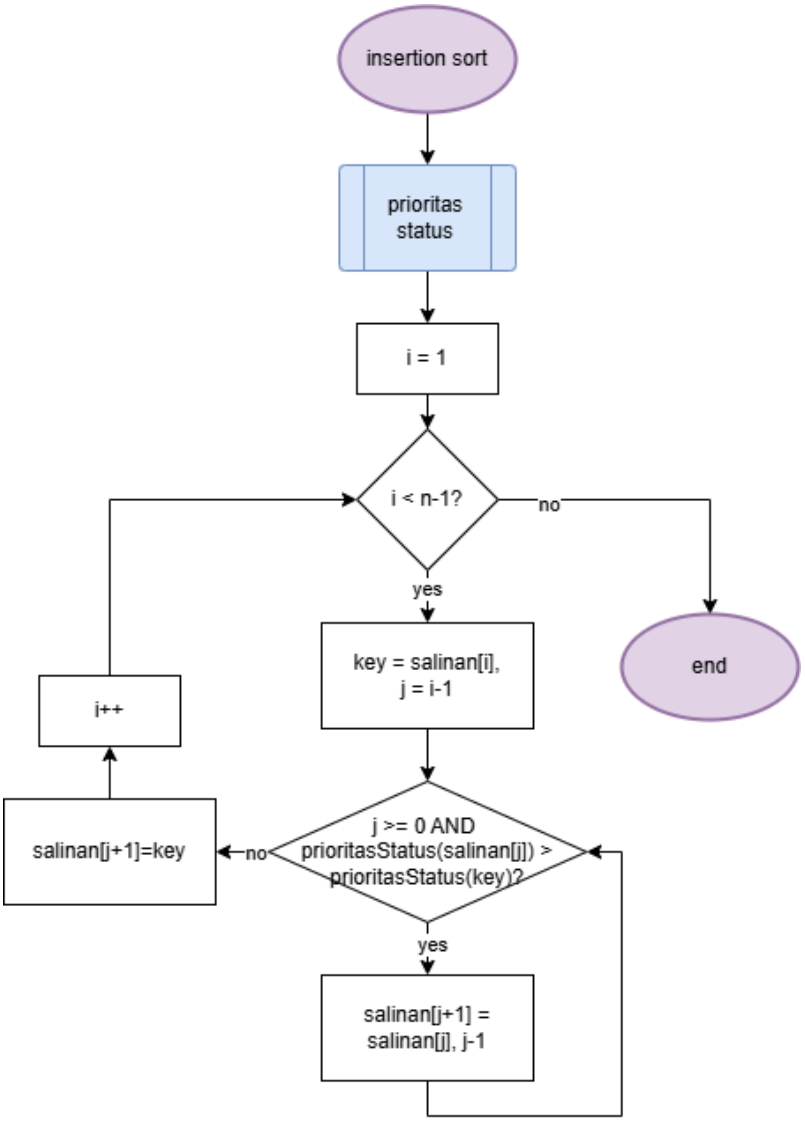
Pada pilihan *sorting* pertama, program akan mengurutkan salinan dari data surat berdasarkan ID surat. Adapun metode yang digunakan adalah *selection sort* secara *ascending.*

Metode ini mencari elemen terkecil terlebih dahulu yang menggunakan elemen pertama sebagai pembanding untuk mencari elemen terkecil, setelah itu maka akan melakukan *increment* jika syarat sudah terpenuhi.



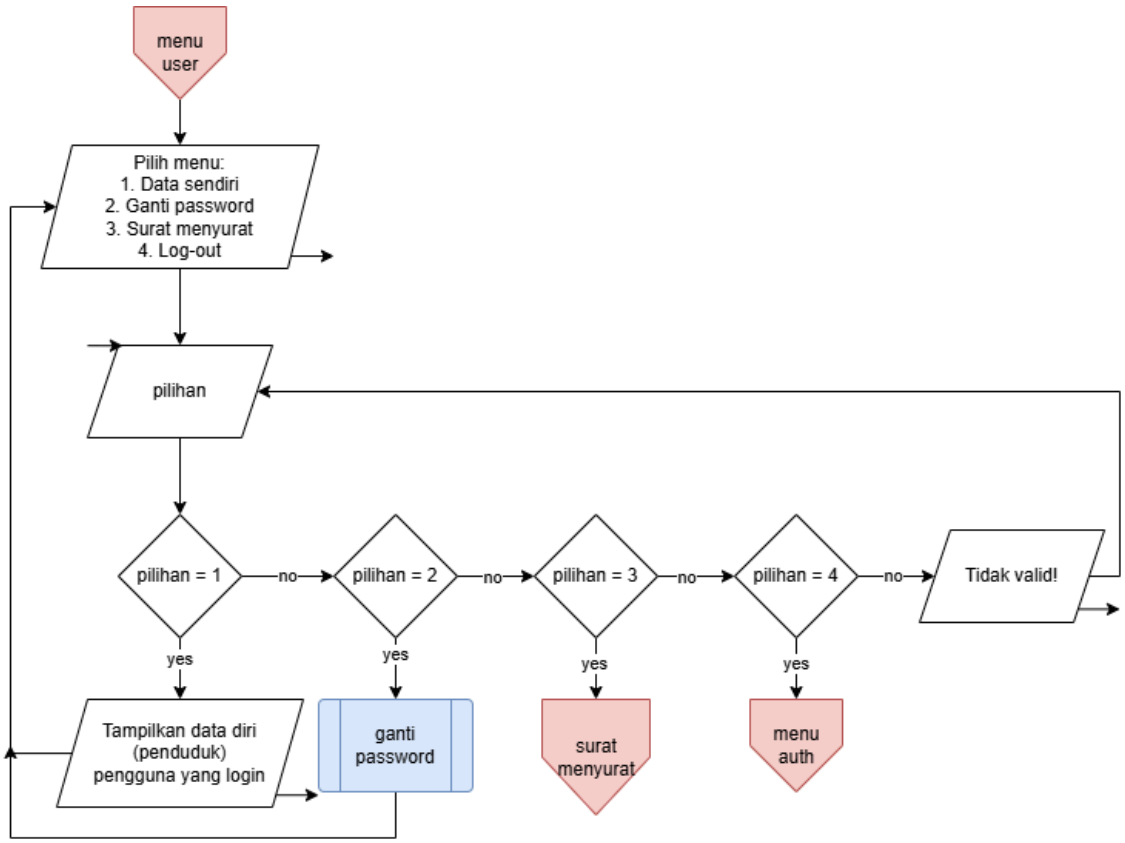
Gambar 1.22 Fungsi Prioritas Surat Status

Fungsi ini bertujuan untuk membaca status suatu surat yang akan digunakan pengurutan berikutnya. Surat disusun berdasarkan prioritas seperti pada *flowchart* di atas. Karena bakal aneh jadinya jika yang diurutkan adalah *string*, maka fungsi ini mengembalikan nilai *integer* sesuai status.



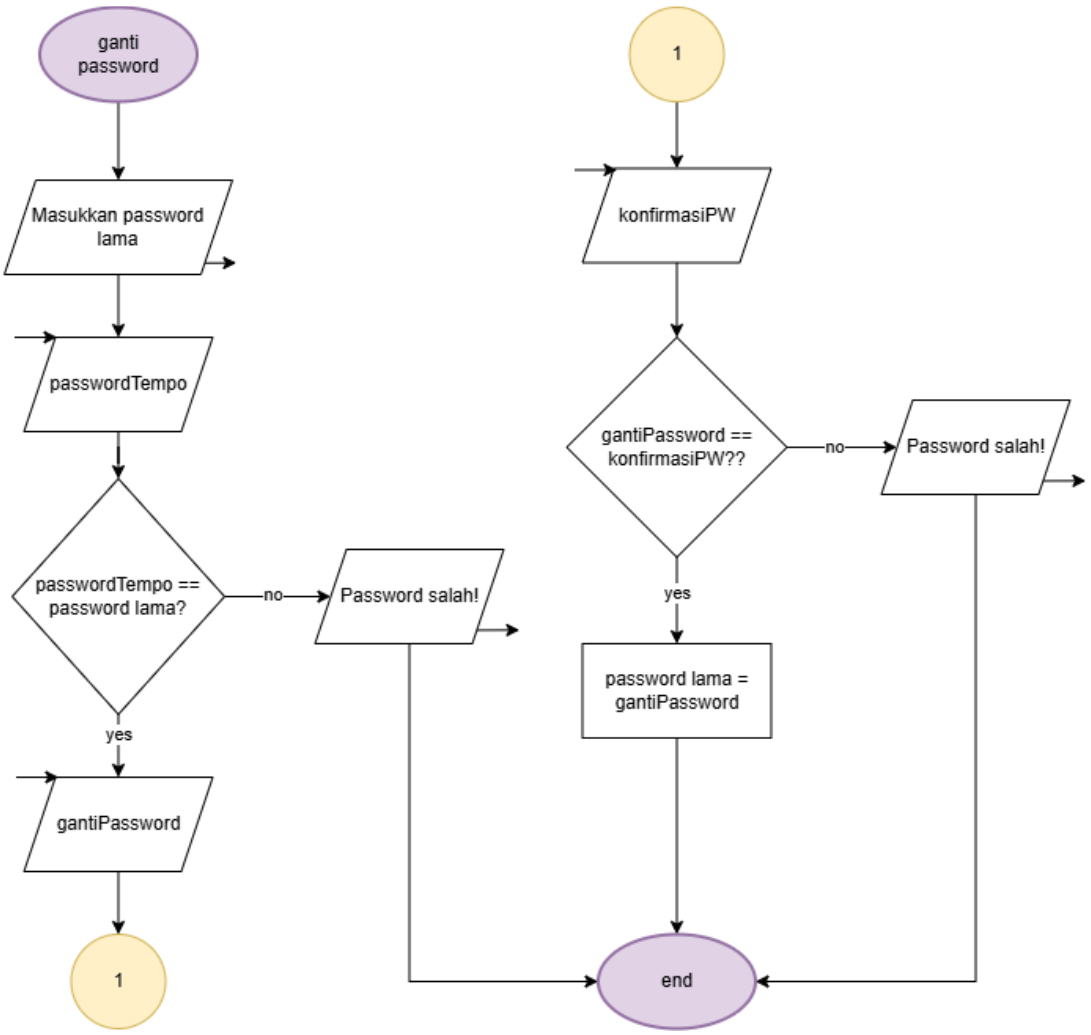
Gambar 1.23 Insertion Sort

Setelah membuat fungsi prioritas, kita buat fungsi *insertion sort*. Metode ini membandingkan dan mengurutkan dua data pertama pada *array*, kemudian membandingkan data berikutnya apakah sudah berada di tempat semestinya.



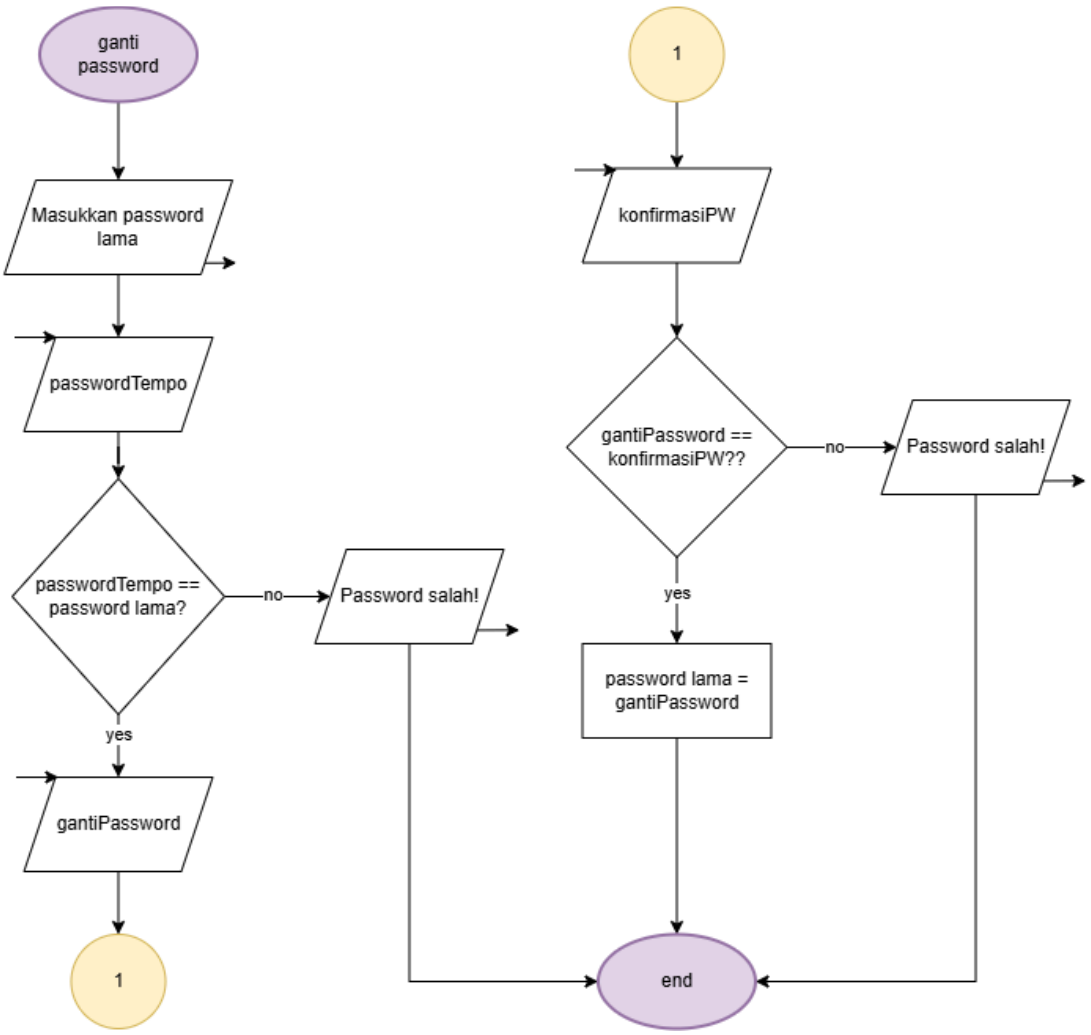
Gambar 1.24 Menu User

Selain admin, ada juga *role user*. *Role* ini hanya bisa memanipulasi data yang terikat dengan dirinya sendiri dan memiliki beberapa batasan. User punya 3 menu utama, menampilkan data mereka, mengganti *password*, dan yang paling utama menu surat menyurat.



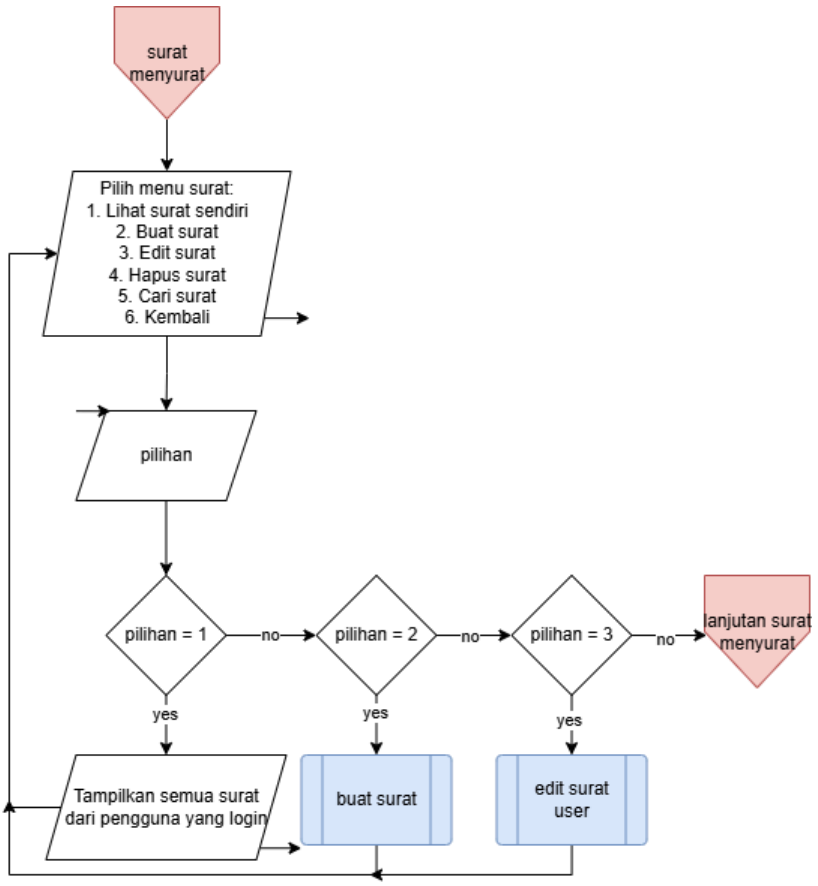
Gambar 1.25 Fungsi Mengganti Password

Untuk menjaga keamanan ketika mengganti *password*, *user* harus memasukkan *password* mereka saat ini. Setelah verifikasi selesai barulah *user* bisa memasukkan *password* baru, adapun *user* juga harus mengkonfirmasi *password* baru mereka dengan memasukkan password yang sama sekali lagi. Jika semua langkah di atas berjalan dengan tepat dan benar, barulah *password* lama akan ditimpa dengan yang baru.

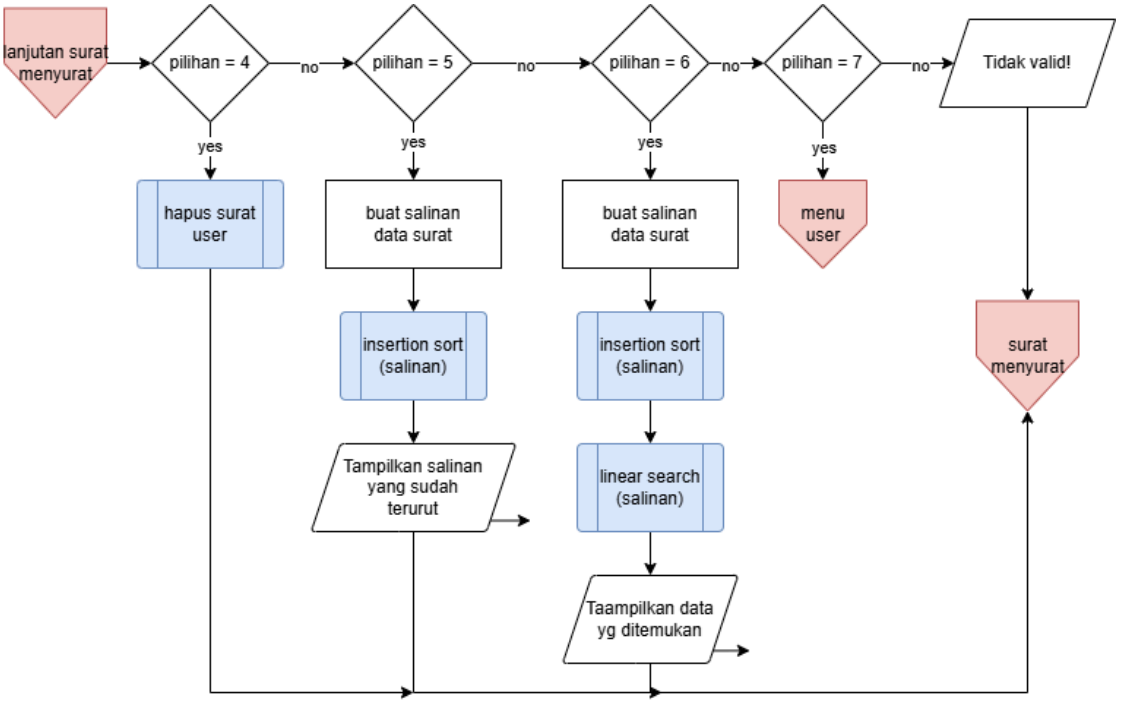


Gambar 1.26 Fungsi Mengganti Password

Untuk menjaga keamanan ketika mengganti *password*, *user* harus memasukkan *password* mereka saat ini. Setelah verifikasi selesai barulah *user* bisa memasukkan *password* baru, adapun *user* juga harus mengkonfirmasi *password* baru mereka dengan memasukkan password yang sama sekali lagi. Jika semua langkah di atas berjalan dengan tepat dan benar, barulah *password* lama akan ditimpa dengan yang baru.



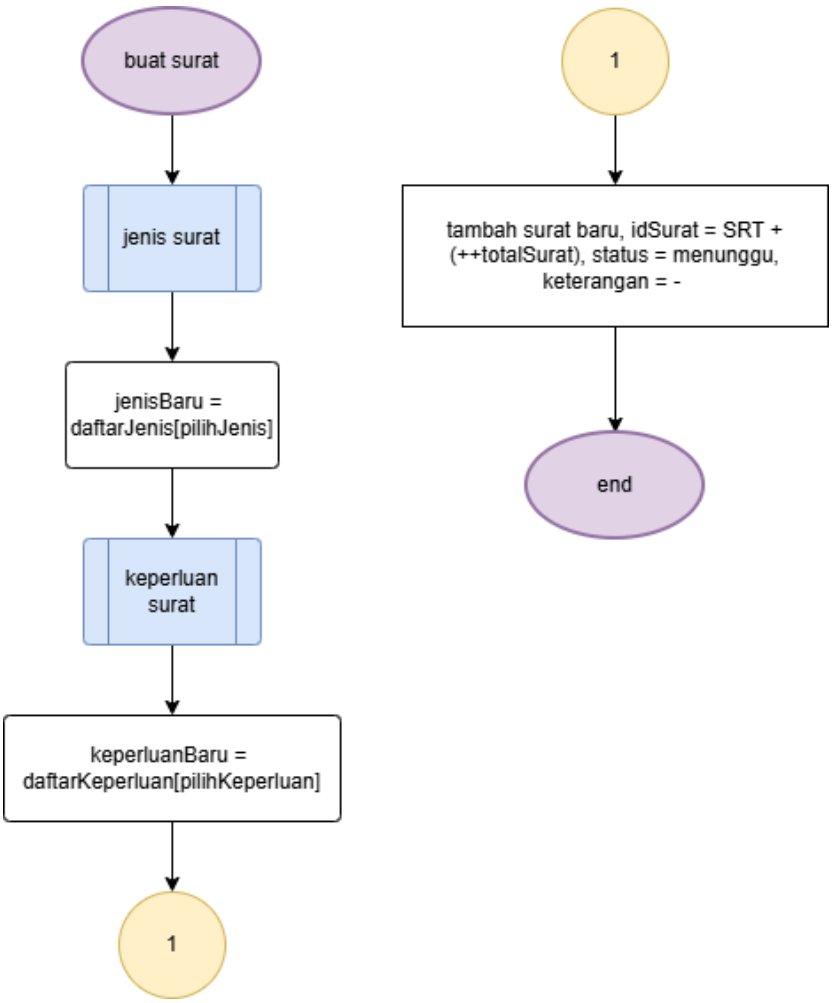
Gambar 1.27 Menu Surat User



Gambar 1.28 Lanjutan Menu Surat User

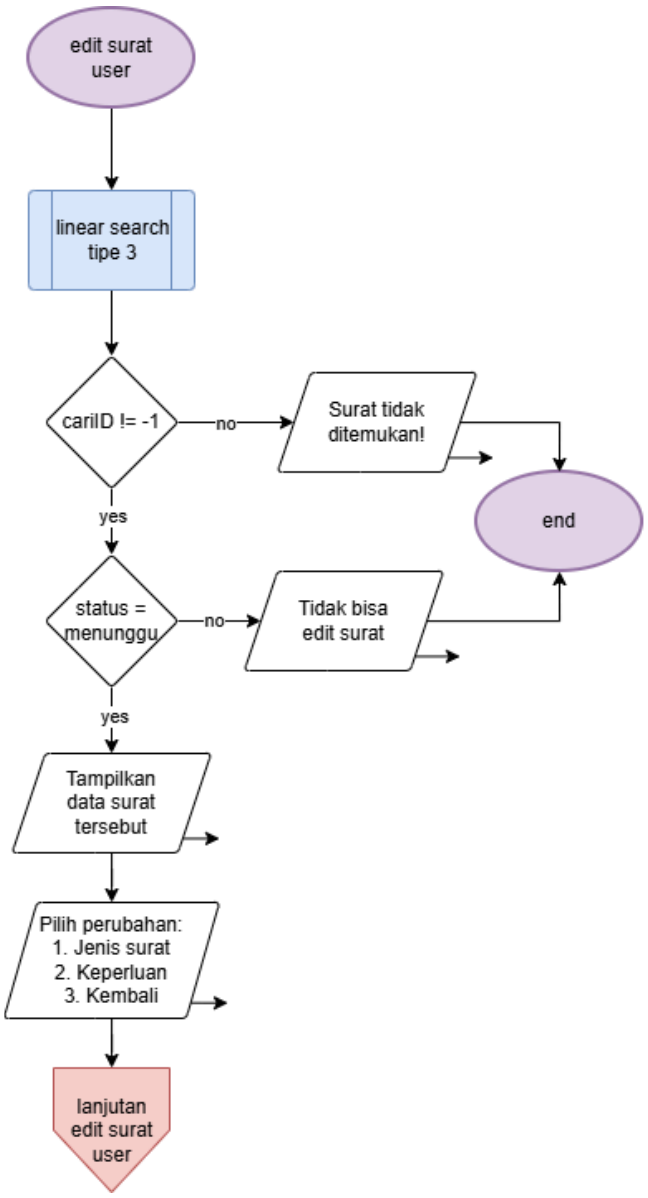
Disini menu utama *user*, mereka secara bebas dapat membuat surat, melihat semua surat yang pernah mereka buat/mohon (pilihan 1), mengedit surat, menghapus surat, dan

mengurutkan surat berdasarkan prioritas status sama seperti pada menu surat admin serta mencari surat mereka dengan *linear search*.



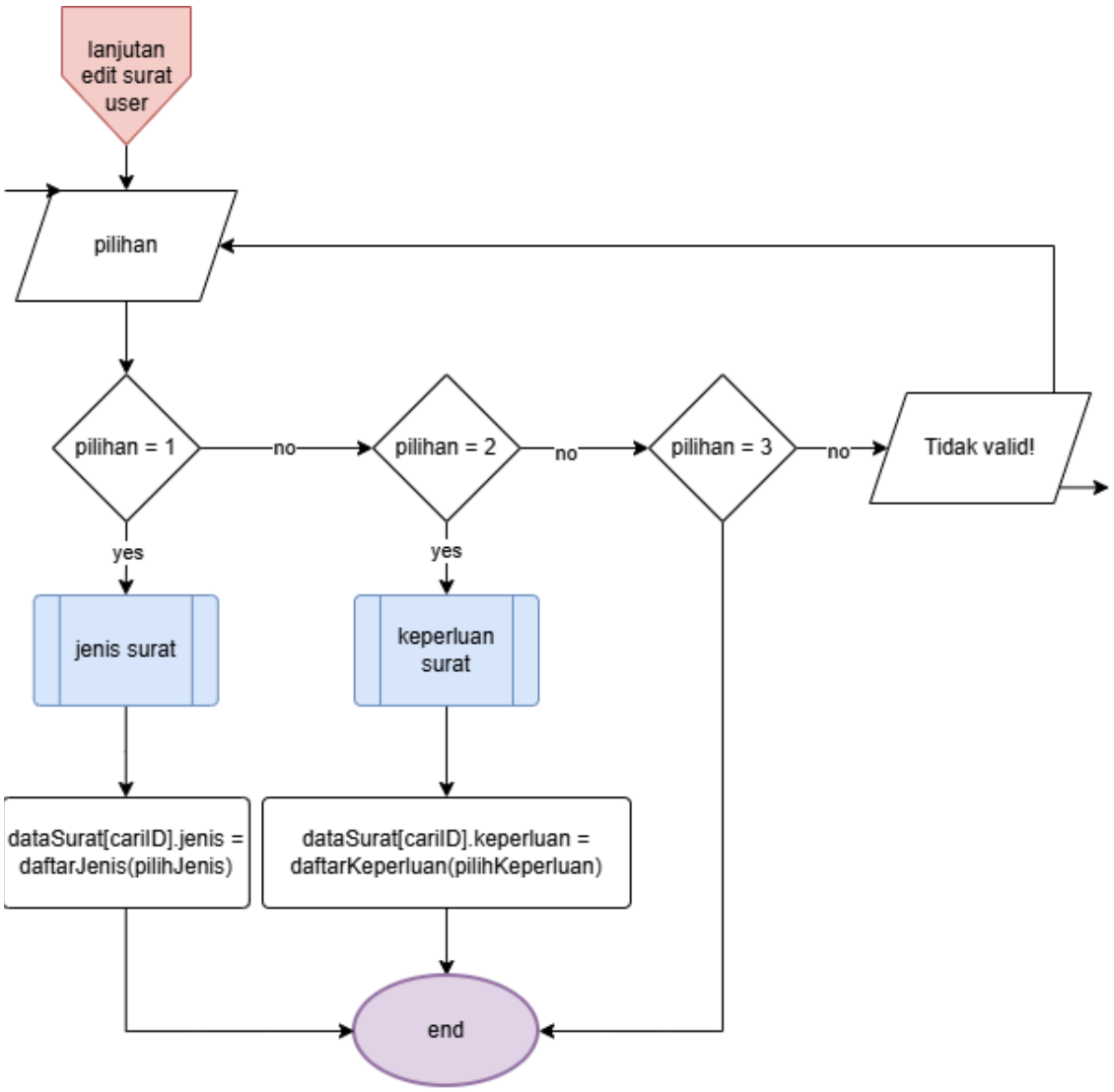
Gambar 1.29 Fungsi Buat Surat

Dalam membuat sura, *user* hanya bisa memilih jenis dan keperluan surat sesuai daftar. Ketika semua sudah selesai, barulah program menambah surat tersebut. ID surat otomatis bertambah seperti NIK, status akan ditetapkan ke menunggu dan isi keterangan kosong yang nantinya akan diubah admin.



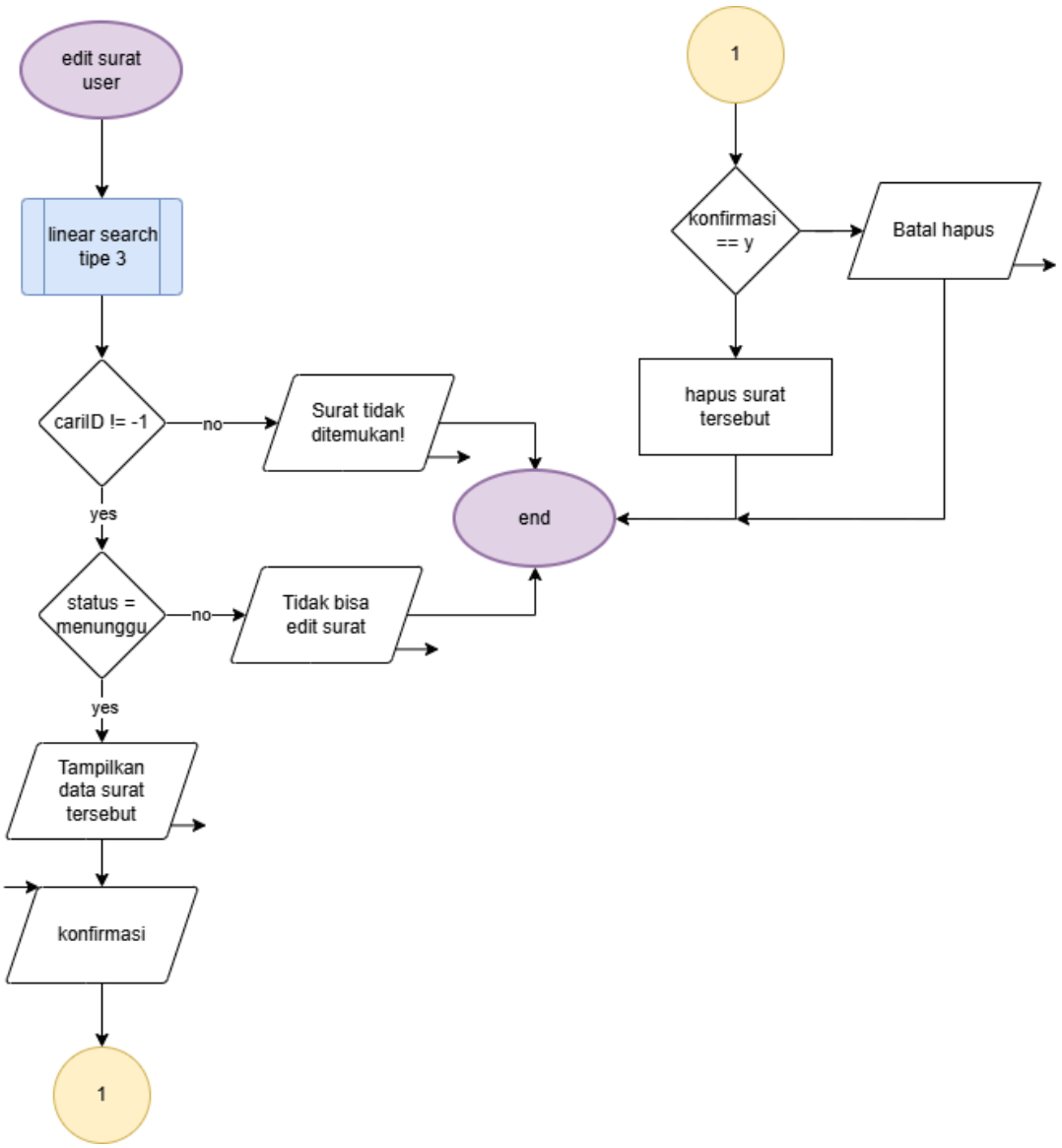
Gambar 1.30 Mengedit Surat Versi User

Sama halnya ketika admin ingin mengedit surat, *user* juga harus memasukkan ID surat yang ingin diubah. Namun disini ada persyaratan, *user* hanya dapat mengedit surat selama status surat tersebut masih menunggu, selebihnya akan ditolak program.



Gambar 1.31 Lanjutan Edit Surat User

User hanya mengedit jenis surat dan keperluan surat menggunakan fungsi pada *Gambar 1.15*. Nilai yang didapat dari fungsi kemudian menimpa data lama.



Gambar 1.32 Hapus Surat Versi User

Proses sama saat mengedit surat, surat hanya bisa dihapus jika status masih menunggu. Setelah *user* mengkonfirmasi penghapusan barulah surat tersebut dihapus sepenuhnya.

2. Deskripsi Singkat Program

Program administrasi balai kota dikelola oleh sekumpulan admin yang memegang database penduduk-penduduknya. Mereka dapat memanipulasi database tersebut. Selain itu mereka juga mengatur akun-akun non-admin. Tujuan utama program ini yaitu kegiatan surat menyurat, user atau pengguna biasa dibebaskan dalam permohonan surat yang ditujukan ke administrasi balai kota. Admin tidak bisa mengubah keaslian surat-surat tersebut melainkan hanya membantu si pemohon dengan terus memperbarui status dan keterangan yang ada pada surat.

3. Source Code

Pada bagian ini alur program dalam *source code* akan dijelaskan, mulai dari variabel, fungsi, hingga fungsi *main*. Penjelasan akan dibagi menjadi beberapa kategori sesuai tipe fungsi dan prosedur yang ada.

Perbedaan dengan posttest sebelumnya yaitu penerapan *Error Handling* menggunakan *exception handling* di beberapa fungsi yang ada, bertujuan untuk memastikan program tetap jalan meski terjadi kesalahan. *Exception* terdiri dari 3 elemen: (1). Try: Blok untuk menempatkan kode yang berpotensi menimbulkan *error*. (2). Throw: Digunakan untuk melempar *error* saat *error* terjadi. (3). Catch: Blok untuk menangkap dan menangani *error* yang dilempar *throw.*

3.1 Library dan Variabel Global

```
#include <iostream>
#include <tabulate/table.hpp>
#include <string>
#include <cmath>
#include <algorithm>
#include "rang.hpp"
using namespace rang;
using namespace tabulate;
using namespace std;

struct Penduduk {
    string nik;
    string nama;
    string alamat;
    string noTelp;
};
struct User {
    string usr;
```

```
    string pw;
    string role;
    Penduduk dataDiri;
};
struct Surat {
    string idSurat;
    string nikPemohon;
    string jenisSurat;
    string keperluan;
    string status;
    string keterangan;
};
string daftarJenis[] {
    "Surat Keterangan Domisili",
    "Surat Keterangan Tidak Mampu",
    "Surat Pengantar KTP",
    "Surat Keterangan Kelahiran"
};
string daftarKeperluan[] {
    "Melamar pekerjaan",
    "Keperluan pendidikan",
    "Keperluan pernikahan",
    "Lainnya"
};
    string daftarStatus[] {
    "menunggu",
    "diproses",
    "selesai",
    "ditolak"
};
```

Source Code 3.1.1 Daftar Library dan Variabel Global

Sebelum masuk ke program, mari bedah awalan program. Disini kita memiliki 3 *struct* utama yaitu 'Penduduk', 'User' dan 'Surat' yang berisi *field-field* pembangun yang akan digunakan untuk CRUD program. *Struct* 'User' menggunakan *nested struct* pada 'Penduduk' yang disimpan di 'dataDiri'.

Lalu ada 3 *array of string* yang berisi nilai dari 3 *field* 'Surat' yaitu 'jenisSurat', 'keperluan', dan 'status'. Nilai ini memang sudah ditetapkan untuk memudahkan program dan mencegah isi pengajuan surat berisi hal tidak yang perlu. Selain itu program ini menggunakan *library* Tabulate untuk mempercantik tampilan *output* program.

Program juga menggunakan *library* Rang, *library* yang memberi warna, gaya, dan latar pada output di terminal. Untuk memberi warna ketik "fg::(warna yang diinginkan)", untuk gaya "style::(gaya yang diinginkan)".

3.2 Daftar Fungsi Minor dan Fungsi Validasi

Fungsi-fungsi di bawah ini bertipekan fungsi validasi yang membantu dalam proses penginputan program. Bertujuan mencegah adanya kesalahan karakter atau input di beberapa bagian.

```
int tampilMenu(string judul, string opsi[], int jumlah) {
    int pilih;
    Table table;
    table.add_row({judul});
    for (int i=0; i < jumlah; i++) {
        table.add_row({to_string(i+1) + ". " + opsi[i]});
    }
    cout << table << "\n";
    cout << "Pilih: ";
    while (!(cin >> pilih) || pilih < 1 || pilih > jumlah) {
        cin.clear();
        cin.ignore(100, '\n');
        cout << fg::red << "Tidak valid!" << style::reset <<"\n";
    }
    return pilih;
```

Source Code 3.2.1 Fungsi Menampilkan Daftar Menu

Program ini berisi banyak CRUD yang menyebabkan banyak blok kode untuk menampilkan *output* menu dan pilihan pengguna. Oleh karena itu mari gunakan fungsi 'tampilMenu' ini. Fungsi ini punya 3 parameter, salah satunya ada 'opsi[]', parameter ini berisi *array of string* yang harus dibuat sebelum memanggil fungsi, adapun parameter 'jumlah' merupakan jumlah dari pilihan menu.

Fungsi ini juga menggunakan tabel dari Tabulate, pertama deklarasi tipe data 'pilih' dan panggil tabel dengan 'Table table'. *Add_row* berfungsi untuk menambah baris tabel diikuti parameter 'judul'. Kemudian lakukan perulangan *for* dari *array of struct* yang diargumenkan dan gabung dengan *add_row*. Penggunaan *to_string* untuk menampilkan angka dengan menambahkan indeks saat ini dengan 1.

Perulangan *while* berguna sebagai *error handling* kalau pengguna memasukkan pilihan kurang atau lebih dari jumlah. Ketika pengguna memasukkan input selain angka maka program langsung meminta pengguna input ulang dan notifikasi kesalahan input. Ketika 'cin' gagal membaca ia akan beku dan 'cin.clear()' yang mengembalikannya ke semula. Adapun 'cin.ignore()' akan membuat sisa karakter pada buffer, maks 100 karakter atau saat membaca enter/newline. Terakhir kembalikan nilai 'pilih'.

```
bool konfirmasi(string pesan) {
```

```
    char jawaban;
    cout << pesan << " (y/n): ";
    cin >> jawaban;
    return jawaban == 'y';
```

Source Code 3.2.2 Fungsi Meminta Konfirmasi

Fungsi ini meminta konfirmasi dari dua karakter. Digunakan pada permintaan konfirmasi sebelum menghapus suatu data. Karakter ‘y’ menjadi nilai *true* dari fungsi ini.

```
string inputStr(string pesan) {
    string hasil;
    while (true) {
        cout << pesan;
        getline(cin, hasil);
        if (hasil != "") return hasil;
        cout << fg::red << "Tidak boleh kosong!"<< style::reset <<"\n";
```

Source Code 3.2.3 Fungsi Input Kosong

Fungsi ini mencegah pengguna memasukkan inputan kosong. Digunakan di berbagai bagian yang meminta input seperti saat menambah data. Kerjanya bandingkan inputan dengan *string* kosong (“”), jika iya maka tidak kembalikan nilai.

```
string inputMax(string pesan, int maxLen) {
    string hasil;
    while (true) {
        cout << pesan;
        getline(cin, hasil);
        if (hasil == "") {
            cout << fg::red << "Tidak boleh kosong!"<< style::reset <<"\n";
        } else if (hasil.length() > maxLen) {
            cout << fg::red << "Tidak boleh lebih dari " << maxLen << "
karakter!" << style::reset << "\n";
        } else {
            return hasil;
        }
    }
}
string inputMin(string pesan, int minLen) {
    string hasil;
    while (true) {
        cout << pesan;
        getline(cin, hasil);
        if (hasil == "") {
            cout << fg::red << "Tidak boleh kosong!"<< style::reset <<"\n";
        } else if (hasil.length() < minLen) {
            cout << fg::red << "Tidak boleh kurang dari " << minLen << "
karakter!" << style::reset << "\n";
        } else {
```

```
            return hasil;
```

Source Code 3.2.4 Fungsi Jumlah Karakter

Fungsi ini berguna untuk mencegah jumlah karakter dari suatu inputan melebihi atau kurang dari batas yang diberikan. Prosesnya gunakan ‘hasil.length()’ untuk menghitung panjang inputan, bandingkan dengan ‘maxLen’/’minLen’.

```
string hanyaAngka(string pesan) {
    string hasil;
    while (true) {
        cout << pesan;
        getline(cin, hasil);
        if (hasil == "") {
            cout << fg::red << "Tidak boleh kosong!"<< style::reset <<"\n";
            continue;
        }
        bool valid = true;
        for (int i = 0; i < hasil.length(); i++) {
            if (!isdigit(hasil[i])) {
                valid = false;
                break;
            }
        }
        if (!valid) {
            cout << fg::red << "Hanya boleh angka!"<< style::reset <<"\n";
        } else {
            return hasil;
```

Source Code 3.2.5 Fungsi Menerima Angka

Fungsi ini digunakan saat meminta inputan nomor telepon, mencegah adanya karakter huruf dan simbol di dalamnya sekaligus mencegah input kosong. Prosesnya menggunakan *flag* ‘valid’ dan *isdigit* untuk membaca karakter angka di dalamnya.

```
string inputOpsional(string pesan, int maxLen) {
    string hasil;
    while (true) {
        cout << pesan;
        getline(cin, hasil);
        if (hasil.length()>maxLen) {
           cout << fg::red << "Tidak boleh lebih dari " << maxLen << "
karakter!" << style::reset << "\n";
        } else {
            return hasil;
```

Source Code 3.2.6 Fungsi Input Kosong Opsional

Fungsi ini digunakan saat mengisi *field* keterangan pada *struct* ‘Surat’. Karena pada *field* itu nilai kosong tidak dilarang.

```
bool statusMenunggu(Surat data[], int index) {
    return data[index].status == "menunggu";
```

Source Code 3.2.7 Fungsi Status Surat

Fungsi ini bertujuan untuk membaca status dari suatu surat, digunakan dalam proses pengeditan surat dan penghapusan surat.

3.3 Daftar Fungsi Searching

```
int linearSearch(void* data, int jumlah, string kunci, int tipe) {
    if (tipe == 1) {
        Penduduk* arr = (Penduduk*)data;
        for (int i=0; i<jumlah; i++) {
            if ((arr+i)->nik == kunci) {
                return i;
            }
        }
    }
    else if (tipe == 2) {
        User* arr = (User*)data;
        for (int i=0; i<jumlah; i++) {
            if((arr+i)->usr == kunci) {
                return i;
            }
        }
    }
    else if (tipe == 3) {
        Surat* arr = (Surat*)data;
        for (int i=0; i<jumlah; i++) {
            if ((arr+i)->idSurat == kunci) {
                return i;
            }
        }
    }
    return -1;
```

Source Code 3.3.1 Linear Search

Blok kode ini mencari data (parameter 'kunci') menggunakan *linear search*. Untuk menyederhanakan program, fungsi menggunakan parameter 'tipe' untuk menentukan data apa yang sedang dicari, penduduk, akun, atau surat.

Selain itu fungsi ini menggunakan parameter *void pointer*, yaitu *pointer* universal agar bisa menunjuk tipe data apa saja. Ini memungkinkan kita tiga *struct* utama. *Linear search* sangat sederhana, membandingkan data satu per satu dengan kunci. Jika ketemu *return* indeksnya jika tidak *return* -1 sebagai operan.

```
int jumpSearch(Penduduk* arr, int jumlah, string kunci) {
    int step = sqrt(jumlah);
    int prev = 0;

    while((arr+min(step, jumlah)-1)->nama > kunci) {
        prev = step;
        step += sqrt(jumlah);
        if (prev >= jumlah) {
            cout << fg::red << "Penduduk tidak ditemukan!"<< style::reset
<<"\n";
            return -1;
        }
    }
    while((arr+prev)->nama > kunci) {
        prev++;
        if (prev == min(step, jumlah)) {
            cout << fg::red << "Penduduk tidak ditemukan!"<< style::reset
<<"\n";
            return -1;
        }
    }
    if ((arr+prev)->nama == kunci) {
        return prev;
    }
    return -1;
```

Source Code 3.3.2 Jump Search

Karena telah diurutkan secara *descending*, maka menggunakan operator lebih dari di perulangan *while*. *Jump search* membagi *array* dengan istilah blok. Blok didapat dengan mengakarkan jumlah elemen pada *array*, yang dimana disimpan di variabel 'step', sementara 'prev' adalah variabel yang menandakan posisi awal blok.

Loop pertama adalah bagian *jump* atau lompat. Pada bagian ini fungsi mengecek elemen terakhir pada blok ini, bandingkan dengan kunci, jika indeks lebih besar dari kunci, maka pindah ke blok berikutnya. Jika pada *loop* ini 'prev' melebihi jumlah maka langsung kembalikan -1 atau tidak ketemu.

Jika nama saat ini melebihi kunci maka kunci ada di antara indeks sebelumnya dan indeks sekarang. Maka dimulai *linear search* dari 'prev', tambah 'prev' sampai ketemu, kembalikan nilainya. Jika melebihi jumlah, maka return -1.

3.4 Daftar Fungsi Autentikasi

Fungsi-fungsi di bawah ini merupakan fungsi dalam proses autentikasi seperti *login* dan registrasi.

```
bool login(User dataUser[], int jumlahUser, int kesempatan, int &indexUser)
{
    try {
        if (kesempatan == 0)
            throw runtime_error("Kesempatan login sudah habis!");
        string username, password;
        cout << "Username: ";
        getline(cin, username);
        cout << "Password: ";
        getline(cin, password);
        for (int i = 0; i < jumlahUser; i++) {
            if (username == dataUser[i].usr && password == dataUser[i].pw) {
                indexUser = i;
                cout << fg::green << "Login berhasil! Selamat datang, " <<
username << "!" << style::reset<<"\n";
                return true;
            }
        }
        cout << fg::yellow << "Salah! Sisa kesempatan: " << kesempatan - 1
<< style::reset << "\n";
        return login(dataUser, jumlahUser, kesempatan - 1, indexUser);
    } catch (runtime_error& e) {
        cout << fg::red << style::bold << "[ERROR]" << e.what() <<
style::reset << "\n";
        return false;
```

Source Code 3.4.1 Fungsi Login

Fungsi ‘login’ di atas menggunakan mekanisme rekursif yaitu fungsi yang memanggil dirinya sendiri dan memerlukan suatu kondisi berhenti. Fungsi ini menggunakan *pass by reference*, yaitu parameter dengan simbol & pada parameter ‘indexUser’, dengan ini setiap perubahan yang terjadi akan mempengaruhi variabel aslinya.

Jika ‘kesempatan’ menyentuh 0, maka kode melempar *error* bertipe ‘runtime_error’ dengan pesan yang ada. Kemudian blok *catch* menangani jenis *error* yang sesuai dan menampilkan pesan melalui ‘e’ dengan pesan yang ada ‘what()’.

Sebelum masuk ke fungsi registrasi, ada beberapa fungsi pendukung yang membantu saat registrasi. Registrasi **hanya bisa dilakukan** jika data penduduk dari pengguna yang mau regis terdaftar dalam data penduduk dan tidak memiliki akun.

```
bool usernameAda (User data[], int jumlah, string username) {
    for (int i=0; i < jumlah; i++) {
        if (data[i].usr == username) return true;
    }
    return false;
```

Source Code 3.4.2 Fungsi Username Terduplikat

Pengguna baru akan ditolak jika memasukkan *username* yang sudah digunakan oleh pengguna lain. Prosesnya lakukan *for* di data 'User' untuk membandingkan inputan *username* dari pengguna yang sedang regis.

```
int cariPendudukDariNama (Penduduk data[], int jumlah, string nama, int
hasilCari[]) {
    int jumlahHasil = 0;
    for (int i=0; i < jumlah; i++) {
        if (data[i].nama == nama) {
            hasilCari[jumlahHasil] = i;
            jumlahHasil++;
        }
    }
    return jumlahHasil;
```

Source Code 3.4.3 Fungsi Cari Nama Penduduk

Bagaimana jika ditemukan beberapa data penduduk dengan nama sama dan semuanya belum memiliki akun? Selain meminta *username* dan *password*, pengguna juga harus memasukkan nama lengkap. Setelah input selesai dilakukan *for* dalam data 'Penduduk'. Setiap ditemukan nama sama, indeks dari penduduk dengan nama tersebut akan disimpan di array 'hasilCari'. Setiap ketemu nama sama 'jumlahHasil' bertambah. Terakhir *array* 'hasilCari' dan 'jumlahHasil' dikembalikan.

```
bool punyaAkun (User dataUser[], int jumlahUser, string nik) {
    for (int i=0; i < jumlahUser; i++) {
        if (dataUser[i].dataDiri.nik == nik) return true;
    }
    return false;
```

Source Code 3.4.4 Fungsi Penduduk Punya Akun

Sebelumnya dijelaskan hanya data penduduk yang belum punya akun yang hanya bisa registrasi. Alurnya lakukan *for* dengan melihat *nested struct* 'dataDiri.nik' dengan NIK yang ingin dibandingkan.

```
void registarsi(User dataUser[], int &jumlahUser, Penduduk dataPenduduk[],
int jumlahPenduduk) {
    try {
        string usernameBaru = inputStr("Username: ");
        if (usernameAda(dataUser, jumlahUser, usernameBaru)) {
```

```
            throw invalid_argument("Username sudah digunakan!");
        }
        string passwordBaru = inputMin("Password: ", 3);

        string namaCari = inputStr("Nama lengkap: ");
        int hasilCari[100];
        int jumlahHasil = cariPendudukDariNama(dataPenduduk, jumlahPenduduk,
namaCari, hasilCari);

        int filter[100];
        int jumlahFilter = 0;
        for (int i=0; i < jumlahHasil; i++) {
            if (!punyaAkun(dataUser, jumlahUser,
dataPenduduk[hasilCari[i]].nik)) {
                filter[jumlahFilter++] = hasilCari[i];
            }
        }

        if (jumlahFilter == 0) {
            throw invalid_argument("Nama tidak ditemukan atau sudah punya
akun!");
        }
        int indexPenduduk;
        if (jumlahFilter == 1) {
            indexPenduduk = filter[0];
        } else {
            Table table;
            table.add_row({"No", "Nama", "Alamat", "NIK"});
            for (int i = 0; i < jumlahFilter; i++) {
                table.add_row({
                    to_string(i+1),
                    dataPenduduk[filter[i]].nama,
                    dataPenduduk[filter[i]].alamat,
                    dataPenduduk[filter[i]].nik
                });
            }
            table.column(0).format().width(4); // No
            table.column(1).format().width(16); //Nama
            table.column(2).format().width(16); // Alamat
            table.column(3).format().width(6); // NIK
            cout << table << "\n";

            int pilih;
            cout << "Pilih nomor: ";
            while (!(cin >> pilih) || pilih < 1 || pilih > jumlahFilter) {
                cin.clear();
                cin.ignore(100, '\n');
                cout << "Tidak valid!\n Pilih: ";
```

```
            }
            indexPenduduk = filter[pilih-1];
        }
        dataUser[jumlahUser++] = {usernameBaru, passwordBaru, "user",
dataPenduduk[indexPenduduk]};
        cout << "NIK kamu: " << dataPenduduk[indexPenduduk].nik << "\n";
        cout << fg::green << "Registrasi berhasil!"<< style::reset << "\n";
    } catch (invalid_argument& e) {
        cout << fg::red << style::bold << "[INPUT TIDAK VALID]" << e.what()
<< style::reset << "\n";
```

Source Code 3.4.5 Prosedur Registrasi

Kita masuk pada prosedur registrasi. Jika fungsi 'usernameAda' *true* maka pengguna kembali ke menu autentikasi. Jika tidak lanjut masukkan *password*, syaratnya tidak boleh kurang dari 3 karakter dan masukkan nama lengkap.

Panggil fungsi 'cariPendudukDariNama' untuk menyimpan setiap nama sama ke *array* 'hasilCari'. Lanjut ke bagian filter mereka yang belum punya akun. Perulangan dilakukan di *array* 'hasil', panggil fungsi 'adaAkun' yang setiap kali mengembalikan nilai *false*, maka data penduduk tersebut masuk ke *array* 'filter', 'jumlahFilter' bertambah satu setiap nilai yang ditemukan.

Jika tidak ditemukan maka artinya nama tidak ditemukan atau semua penduduk dengan nama tersebut sudah memiliki akun. Jika hanya menemukan satu data maka bisa langsung disimpan di variabel 'indexPenduduk'. Jika ditemukan data lebih dari satu, semua data ditampilkan dalam tabel beserta *field* lainnya. Pengguna harus memilih data yang sesuai. Terakhir akun baru dibuat dengan *role* bawaan yaitu *user* dan *nested struct* dari 'dataPenduduk[indexPenduduk]'.

Terdapat dua blok *throw* disini, namun karena kedua tipe *error* sama maka hanya perlu satu blok *catch* untuk menanganinya.

3.5 Daftar Fungsi Pendukung Fitur Utama

```
void tampilSuratByNIK(Surat data[], int jumlah, string nik) {
    Table table;
    table.add_row({"No", "ID Surat", "Jenis Surat", "Keperluan", "Status",
"Keterangan"});
    int no = 1;
    bool ada = false;
    for (int i = 0; i < jumlah; i++) {
        if (data[i].nikPemohon == nik) {
            table.add_row({
                to_string(no++),
                data[i].idSurat,
```

```
                data[i].jenisSurat,
                data[i].keperluan,
                data[i].status,
                data[i].keterangan
            });
            ada = true;
        }
    }
    if (!ada) {
        cout << fg::red << "Belum ada surat!" << style::reset <<"\n";
        return;
    }
    table.column(0).format().width(4);
    table.column(1).format().width(6);
    table.column(2).format().width(17);
    table.column(3).format().width(11);
    table.column(4).format().width(8);
    table.column(5).format().width(20);
    cout << table << "\n";
```

Source Code 3.5.1 Prosedur Menampilkan Surat Individu

Prosedur ini berbeda dengan prosedur 'tampilSurat' sebelumnya. Prosedur ini berkhusus untuk akun dengan *role user* yang hanya bisa melihat surat mereka sendiri. Perulangan *for* diikuti dengan *if* yang membandingkan 'nikPemohon' dari *struct* 'Surat' dengan NIK dari pengguna yang sedang *login*. Maka semua surat dengan NIK sesuai akan ditampilkan. Adapun *flag* 'ada' berfungsi untuk mengembalikan pengguna ke menu sebelumnya jika sedang tidak memiliki surat. Selebihnya prosedur ini memiliki proses yang sama dengan prosedur 'tampilSurat'.

```
int cariSuratUser(Surat data[], int jumlah, string idSurat, string nik) {
    for (int i=0; i < jumlah; i++) {
        if (data[i].idSurat == idSurat && data[i].nikPemohon == nik) {
            return i;
        }
    }
    return -1;
```

Source Code 3.5.2 Fungsi Mencari Surat Versi User

Fungsi bekerja persis seperti fungsi *linear search* pada fungsi sebelumnya, hanya saja perbandingan juga dilakukan melalui 'nikPemohon' dan NIK dari pengguna yang sedang

login. Meskipun ID surat itu unik fungsi ini memastikan bahwa setiap pengguna tidak dapat mencari surat dari pengguna lain.

```
void hapus(Penduduk data[], int &jumlah, int index) {
    for (int i= index; i < jumlah - 1; i++) {
        data[i] = data[i+1];
    }
    jumlah--;
}
void hapus(User data[], int &jumlah, int index) {
    for (int i= index; i < jumlah - 1; i++) {
        data[i] = data[i+1];
    }
    jumlah--;
}
void hapus(Surat data[], int &jumlah, int index) {
    for (int i= index; i < jumlah - 1; i++) {
        data[i] = data[i+1];
    }
    jumlah--;
```

Source Code 3.5.3 Prosedur Overloading Hapus

Tiga prosedur ini juga menggunakan mekanisme *overloading*. Prosedur 'hapus' adalah prosedur untuk menghapus suatu data. Parameter 'int &jumlah' menggunakan *address-of operator* '&' agar perubahan nilai jumlah di dalam fungsi langsung mempengaruhi variabel aslinya di fungsi *main*.

Pergeseran dilakukan menggunakan perulangan *for*, indeks dimulai dari indeks *index* kenapa? Kita tidak perlu memulai dari indeks 0 karena indeks yang tepat sudah ditemukan. Jika diperhatikan kondisi perulangan ini adalah jumlah - 1, ini bertujuan untuk mencegah *out of bounds*. Bayangkan pergeseran elemen yaitu elemen kanan mengisi elemen kiri, berarti elemen terakhir tidak ada pengisi jika tidak ada -1. Lalu data ke-i ditimpa dengan data ke-i+1. Terakhir jumlah berkurang, dan karena jumlah dikirim by reference maka pengurangan ini langsung tercermin pada variabel jumlah yang ada di *main*.

3.6 Daftar Fungsi Fitur Utama Admin

```
void tampilPenduduk(Penduduk * data, int jumlah) {
    Table table;
    table.add_row({"No", "NIK", "Nama", "Alamat", "No Telp"});
    for (int i=0; i < jumlah; i++) {
        table.add_row({
            to_string(i+1),
```

```
            (data+i)->nik,
            (data+i)->nama,
            (data+i)->alamat,
            (data+i)->noTelp});
    }
    table.column(0).format().width(4); // No
    table.column(1).format().width(6); // NIK
    table.column(2).format().width(16); // Nama
    table.column(3).format().width(16); // Alamat
    table.column(4).format().width(13); // No Telp

    cout << table << "\n";
```

Source Code 3.6.1 Prosedur Menampilkan Daftar Penduduk

Prosedur ini berisi proses menampilkan daftar penduduk. Pakai parameter *pointer* 'Penduduk * data' yang menunjuk ke elemen pertama *array struct* 'Penduduk'. Lakukan perulangan *for* dan gunakan '(data+i)->field' untuk mengakses setiap elemen, di mana (data+i) adalah pointer *arithmetic* untuk berpindah ke elemen ke-i, dan '->' adalah *dereference operator* untuk mengakses *field* dari *struct* yang ditunjuk. Hasilnya ditampilkan menggunakan tabel dengan 5 kolom. Kegunaan format dan *width* pada bagian bawah fungsi merupakan batas jumlah karakter pada satu baris. Jika melebihi maka lanjut ke baris berikutnya. Ini untuk mencegah tabel menjadi sangat panjang.

```
void tampilAkun(User * data, int jumlah) {
    Table table;
    table.add_row({"No", "Username", "Role", "NIK", "Nama"});
    for (int i=0; i < jumlah; i++) {
        table.add_row({
            to_string(i+1),
            (data+i)->usr,
            (data+i)->role,
            (data+i)->dataDiri.nik,
            (data+i)->dataDiri.nama});
    }
    table.column(0).format().width(4); // No
    table.column(1).format().width(16); // Username
    table.column(2).format().width(5); // Role
    table.column(3).format().width(6); // NIK
    table.column(4).format().width(16); // Nama
    cout << table << "\n";
}

void tampilSurat(Surat * data, int jumlah) {
    Table table;
```

```
    table.add_row({"No", "ID Surat", "NIK Pemohon", "Jenis Surat",
        "Keperluan", "Status", "Keterangan"});
    for (int i=0; i < jumlah; i++) {
        table.add_row({
            to_string(i+1),
            (data+i)->idSurat,
            (data+i)->nikPemohon,
            (data+i)->jenisSurat,
            (data+i)->keperluan,
            (data+i)->status,
            (data+i)->keterangan});
    }
    table.column(0).format().width(4); // No
    table.column(1).format().width(6); // ID Surat
    table.column(2).format().width(6); // NIK Pemohon
    table.column(3).format().width(17); // Jenis Surat
    table.column(4).format().width(11); // Keperluan
    table.column(5).format().width(8); // Status
    table.column(6).format().width(20); // Keterangan
    cout << table << "\n";
```

Source Code 3.6.2 Prosedur Menampilkan Daftar Akun dan Surat

Kedua prosedur ini memiliki proses yang sama dengan prosedur 'tampilPenduduk'. Gunakan *deference operator* untuk menampilkan daftar akun dan surat.

```
void tambahPenduduk(Penduduk data[], int &jumlah, int &totalPenduduk) {
    try {
        string namaBaru, alamatBaru, noTelpBaru;
        namaBaru = inputStr("Nama penduduk: ");
        alamatBaru = inputStr("Alamat penduduk: ");
        noTelpBaru = hanyaAngka("No telepon penduduk: ");
        if (telpAda(data, jumlah, noTelpBaru )) {
            throw invalid_argument("Nomor telepon sudah digunakan!");
        }
        totalPenduduk++;
        string nikBaru = "PDK" + string(totalPenduduk < 10 ? "00" :
totalPenduduk < 100 ? "0" : "") + to_string(totalPenduduk);
        data[jumlah++] = {nikBaru, namaBaru, alamatBaru, noTelpBaru};
        cout << fg::green << "Berhasil ditambahkan!" << style::reset <<
"\n";
    } catch (invalid_argument& e) {
        cout << fg::red << style::bold << "[INPUT TIDAK VALID]" << e.what()
<< style::reset << "\n";
```

Source Code 3.6.3 Prosedur Menambah Penduduk

Simpan semua *field* pada variabel baru, gunakan fungsi ‘inputStr’ untuk mencegah nilai kosong dan ‘hanyaAngka’ pada nomor telepon. Lalu validasi menggunakan fungsi ‘telpAda’.

NIK dari penduduk baru akan otomatis terisi dengan menambah ‘totalPenduduk’, format penamaan NIK adalah “PDK” diikuti mereka penduduk ke berapa. Kita menggunakan *ternary operator* untuk melihat digit ‘totalPenduduk’, jika kurang dari dua digit, maka tambah dua angka nol “00” setelah “PDK” diikuti ‘totalPenduduk’ sekarang. Jika kurang dari tiga digit atau puluhan maka hanya menambah satu nol “0” saja. Harus diingat proses *integer* ini harus diubah ke *string*.

Perbedaan antara jumlah dan total adalah jumlah berisi kuantitas saat ini, sedangkan total adalah kuantitas yang pernah ada, termasuk yang sudah dihapus, ini memungkinkan NIK menjadi *field* unik yang tidak bisa sama satu sama lain. Simpan di data penduduk baru.

```
void editPenduduk(Penduduk dataPenduduk[], int jumlahPenduduk, User
dataUser[], int jumlahUser) {
    try {
        string cariNIK;
        cout << "Masukkan NIK dari data penduduk yang ingin diubah: ";
        getline(cin, cariNIK);
        int cariIndex = linearSearch(dataPenduduk, jumlahPenduduk, cariNIK,
1);
        if (cariIndex == -1) {
            throw out_of_range("NIK tidak ditemukan!");
        }
        cout << "Data ditemukan: \n";
        Table table;
        table.add_row({"NIK", "Nama", "Alamat", "No Telp"});
        table.add_row({
        dataPenduduk[cariIndex].nik,
        dataPenduduk[cariIndex].nama,
        dataPenduduk[cariIndex].alamat,
        dataPenduduk[cariIndex].noTelp
        });
        table.column(0).format().width(6); // NIK
        table.column(1).format().width(16); // Nama
        table.column(2).format().width(16); // Alamat
        table.column(3).format().width(13); // No Telp
        cout << table << "\n";
        string opsiUbahPenduduk[] = {"Ubah nama penduduk", "Ubah alamat
penduduk", "Ubah no telepon penduduk", "Kembali"};
        int pilihan = tampilMenu("Pilih perubahan\n", opsiUbahPenduduk, 4 );
        cin.ignore();
        switch (pilihan) {
            case 1: {
```

```
                string namaGanti = inputStr("Nama baru: ");
                dataPenduduk[cariIndex].nama = namaGanti;
                for (int i=0; i < jumlahUser; i++) {
                    if (dataUser[i].dataDiri.nik ==
dataPenduduk[cariIndex].nik) {
                        dataUser[i].dataDiri.nama = namaGanti;
                        break;
```

Source Code 3.6.4 Prosedur Mengedit Data Penduduk

Prosedur dimulai dengan mencari NIK, dengan fungsi *linear search* tipe 1 atau bagian penduduk. Jika ketemu data penduduk akan ditampilkan dalam tabel dan pengguna pilih perubahan yang diinginkan. Contoh pengguna ingin mengubah nama maka masukkan nama baru kemudian data lama akan ditimpa dengan yang baru. Proses serupa ketika saat menambah penduduk hanya berbeda di kemana data tersebut akan pergi yaitu menimpa data lama. Data pada *struct* 'User' juga diganti mengingat menggunakan *nested struct*. Proses ini juga sama saat mengubah alamat dan nomor telepon.

```
dataPenduduk[cariIndex].alamat = alamatGanti;
if (dataUser[i].dataDiri.nik == dataPenduduk[cariIndex].nik) {
                    dataUser[i].dataDiri.alamat = alamatGanti;
dataPenduduk[cariIndex].noTelp = noTelpGanti;
if (dataUser[i].dataDiri.nik == dataPenduduk[cariIndex].nik) {
                    dataUser[i].dataDiri.noTelp = noTelpGanti;
```

Source Code 3.6.5 Pengeditan Alamat dan Nomor Telepon

```
void hapusPendudukAdmin(Penduduk dataPenduduk[], int &jumlahPenduduk, User
dataUser[], int &jumlahUser, Surat dataSurat[], int &jumlahSurat, int
&indexUser) {
    try {
        string cariNIK;
        cout << "Masukkan NIK dari data penduduk yang ingin diubah: ";
        getline(cin, cariNIK);
        int cariIndex = linearSearch(dataPenduduk, jumlahPenduduk, cariNIK,
1);
        if (cariIndex == -1) {
            throw out_of_range("NIK tidak ditemukan!");
        }
        bool adaAdmin = false;
        for (int i = 0; i < jumlahUser; i++) {
            if (dataUser[i].dataDiri.nik == dataPenduduk[cariIndex].nik &&
            dataUser[i].role == "admin") {
                adaAdmin = true;
                break;
            }
        }
```

```
        if (adaAdmin) {
            throw logic_error("Tidak bisa hapus penduduk dengan akun
admin");
        }
        cout << "Data ditemukan: \n";
        Table table;
        table.add_row({"NIK", "Nama", "Alamat", "No Telp"});
        table.add_row({
        dataPenduduk[cariIndex].nik,
        dataPenduduk[cariIndex].nama,
        dataPenduduk[cariIndex].alamat,
        dataPenduduk[cariIndex].noTelp
        });
        table.column(0).format().width(6); // NIK
        table.column(1).format().width(16); // Nama
        table.column(2).format().width(16); // Alamat
        table.column(3).format().width(13); // No Telp
        cout << table << "\n";
        if (!konfirmasi("Yakin ingin hapus?")) {
            cout << "Batal!\n";
            return;
        }
        string nikHapus = dataPenduduk[cariIndex].nik;
        hapus(dataPenduduk, jumlahPenduduk, cariIndex);

        for (int i=0; i < jumlahUser; i++) {
            if (dataUser[i].dataDiri.nik == nikHapus) {
                if (i<indexUser) indexUser--;
                hapus(dataUser, jumlahUser, i);
                break;
            }
        }

        for (int i=0; i < jumlahSurat; i++) {
            if (dataSurat[i].nikPemohon == nikHapus) {
                hapus(dataSurat, jumlahSurat, i);
                i--;
            }
        }
        cout << fg::green << "Berhasil dihapus!" << style::reset << "\n";
    } catch (out_of_range& e) {
        cout << fg:: red << style::bold << "[TIDAK DITEMUKAN]" << e.what()
<< style::reset << "\n";
    } catch (logic_error& e) {
        cout << fg::red << style::bold << "[PERMINTAAN DITOLAK]" << e.what()
<< style::reset << "\n";
```

Source Code 3.6.6 Prosedur Menghapus Penduduk

Proses penghapusan dimulai dengan mencari NIK, setelah ketemu data penduduk tersebut akan ditampilkan, sama dengan proses pengeditan, pengguna tidak dapat menghapus akun admin. Sebelum menghapus, pengguna diminta konfirmasi penghapusan.

Proses penghapusan dilakukan dengan memanggil prosedur 'hapus' dengan cara pergeseran. Sebelum itu NIK dari data penduduk yang dihapus akan disimpan di variabel 'nikHapus' kenapa? Dalam penghapusan penduduk, akun dan semua surat yang penduduk tersebut miliki juga akan dihapus.

Jika diperhatikan di kode penghapusan akun ada kondisi *if* jika indeks penduduk yang dihapus kurang dari indeks penduduk yang sedang *login*, maka nilai dari variabel 'indexUser' akan berkurang 1, ini untuk mencegah kesalahan indeks karena setelah pergeseran ada kemungkinan indeks penduduk tidak berubah. Penghapusan surat juga ada perintah tambahan yaitu 'i– -', sebelum 'i' pada perulangan *for* bertambah 'i' akan dikurangi terlebih dahulu, ini bertujuan untuk memastikan semua surat dari penduduk tersebut terhapus dan tidak ada yang terlewat.

```
void hapusAkun(User data[], int &jumlah, int &indexUser) {
    try {
        string inputUser;
        cout << "Masukkan username dari akun yang ingin dihapus: ";
        getline(cin, inputUser);
        int cariIndex = linearSearch(data, jumlah, inputUser, 2);
        if (cariIndex == -1) {
            throw out_of_range("Username tidak ditemukan!");
        }
        if (data[cariIndex].role == "admin") {
            throw logic_error("Tidak bisa hapus akun dengan akun admin");
        }
        cout << "Data ditemukan: \n";
        Table table;
        table.add_row({"Username", "Role", "NIK", "Nama"});
        table.add_row({
        data[cariIndex].usr,
        data[cariIndex].role,
        data[cariIndex].dataDiri.nik,
        data[cariIndex].dataDiri.nama
        });
        table.column(0).format().width(16); // Username
        table.column(1).format().width(5); // Role
        table.column(2).format().width(6); // NIK
        table.column(3).format().width(16); // Nama
        cout << table << "\n";
        if (!konfirmasi("Yakin ingin hapus?")) {
            cout << "Batal hapus!\n";
```

```
            return;
        }
        hapus(data, jumlah, cariIndex);
        if (cariIndex < indexUser) {
            indexUser--;
        }
        cout << fg::green << "Berhasil dihapus!" << style::reset << "\n";
    } catch (out_of_range& e) {
        cout << fg::red << style::bold << "[TIDAK DITEMUKAN]" << e.what() <<
style::reset << "\n";
    } catch (logic_error& e) {
        cout << fg::red << style::bold << "[PERMINTAAN DITOLAK]" << e.what()
<< style::reset << "\n";
```

Source Code 3.6.7 Prosedur Menghapus Akun

Proses ini sama persis dengan penghapusan data penduduk, cari akun berdasarkan *username* lalu panggil 'hapus' versi akun. Proses penghapusan sama seperti subbab sebelumnya.

```
void updateSuratAdmin(Surat data[], int jumlah) {
    try {
        string cariID;
        cout << "Masukkan ID surat dari surat yang ingin diubah:";
        getline(cin, cariID);
        int cariIndex = linearSearch(data, jumlah, cariID, 3);
        if (cariIndex == -1) {
            throw out_of_range("Surat tidak ditemukan!");
        }
        cout << "Data ditemukan: \n";
        ...
        cout << table << "\n";
        string opsiUbahSurat[] = {"Ubah status", "Ubah keterangan",
"Kembali"};
        int pilihan = tampilMenu("Pilih perubahan\n", opsiUbahSurat, 3);
        switch(pilihan) {
            case 1: {
                int pilihStatus = tampilMenu("Pilih status\n", daftarStatus,
4); // Tolong dicek lagi
                data[cariIndex].status = daftarStatus[pilihStatus-1];
                cout << fg::green << "Status berhasil diganti!" <<
style::reset << "\n";
                break;
            }
            case 2: {
                cin.ignore();
                string isiKeterangan = inputOpsional("Isi keterangan surat
(kosongkan untuk '-'): ", 50);
                if (isiKeterangan == "") {
```

```
                    data[cariIndex].keterangan = "-";
                    cout << "Keterangan dikosongkan\n";
                    break;
                }
                data[cariIndex].keterangan = isiKeterangan;
                cout << fg::green << "Keterangan berhasil diubah!" <<
style::reset << "\n";
                break;
            }
            case 3: {
                break;
            }
        }
    } catch (out_of_range& e) {
        cout << fg::red << style::bold << "[TIDAK DITEMUKAN]" << e.what() <<
style::reset << "\n";
```

Source Code 3.6.8 Prosedur Mengedit Surat Versi Admin

Untuk melakukan pengeditan surat, cari surat berdasarkan ID, setelah ketemu pengguna dapat memilih perubahan yang diinginkan. Di perubahan status daftar akan ditampilkan menggunakan fungsi 'tampilMenu', pilihan disimpan di dalam 'daftarStatus[pilihStatus-1]' dan dilakukan dengan cara timpa. Perubahan keterangan juga dengan cara timpa. Khusus keterangan, *field* ini boleh dikosongkan dan mengisi nilainya dengan *strip*.

```
void hapusSuratAdmin(Surat data[], int &jumlah) {
    try {
        string cariID;
        cout << "Masukkan ID surat dari surat yang ingin diubah:";
        getline(cin, cariID);
        int cariIndex = linearSearch(data, jumlah, cariID, 3);
        if (cariIndex == -1) {
            throw out_of_range("Surat tidak ditemukan!");
        }
        cout << "Data ditemukan: \n";
            ...
            if(!konfirmasi("Yakin ingin hapus?")) {
            cout << "Batal hapus!\n";
            return;
        }
        hapus(data, jumlah,cariIndex);
        cout << fg::green << "Berhasil dihapus!" << style::reset << "\n";
    } catch (out_of_range& e) {
        cout << fg::red << style::bold << "[TIDAK DITEMUKAN]" << e.what() <<
style::reset << "\n";
```

Source Code 3.6.9 Prosedur Hapus Surat Admin

Pada prosedur ini, kita harus memasukkan ID surat yang ingin dihapus. Surat tersebut kemudian ditampilkan dan program meminta konfirmasi pengguna sebelum menghapus. Jika ya maka surat tersebut dihapus.

3.7 Daftar Fungsi Fitur Utama User

```
void gantiPassword(User data[], int indexUser) {
    try {
        string passwordTempo = inputStr("Masukkan password sekarang: ");
        if (data[indexUser].pw != passwordTempo)
            throw invalid_argument("Password saat ini salah!");

        string passwordGanti = inputMin("Masukkan password baru: ", 3);
        string konfirmasiPw = inputStr("Konfirmasi password baru: ");
        if (passwordGanti != konfirmasiPw)
            throw invalid_argument("Konfirmasi password tidak cocok!");

        data[indexUser].pw = passwordGanti;
        cout << fg::green << "Password berhasil diubah!" << style::reset <<
"\n";

    } catch (invalid_argument& e) {
        cout << fg::red << style::bold << "[INPUT TIDAK VALID] " << e.what()
<< style::reset << "\n";
```

Source Code 3.7.1 Prosedur Mengganti Password

Prosedur ini memungkinkan pengguna dengan *role user* mengganti *password* mereka. Prosesnya masukkan *password* saat ini untuk verifikasi, setelah itu masukkan *password* baru dan masukkan *password* yang sama sekali lagi sebagai konfirmasi. Proses ini menggunakan banyak proses perbandingan.

```
void buatSurat(Surat dataSurat[], int &jumlahSurat, int &totalSurat, int
indexUser, User dataUser[]) {
    int pilihJenis = tampilMenu("Pilih Jenis\n", daftarJenis, 4);
    string jenisBaru = daftarJenis[pilihJenis-1];
    int pilihKeperluan = tampilMenu("Pilih keperluan\n", daftarKeperluan,
4);
    string keperluanBaru = daftarKeperluan[pilihKeperluan-1];
    string nikPemohonBaru = dataUser[indexUser].dataDiri.nik;
    totalSurat++;
    string idBaru = "SRT" + string(totalSurat < 10 ? "00" : totalSurat < 100
? "0" : "") + to_string(totalSurat);
    dataSurat[jumlahSurat] = {idBaru, nikPemohonBaru, jenisBaru,
keperluanBaru, "menunggu", "-"};
    jumlahSurat++;
```

```
    cout << "Surat berhasil dibuat!\n";
```

Source Code 3.7.2 Prosedur Menambah Surat

Prosedur ini dikhususkan ke *user*, pengisian *field* 'jenisSurat' dan 'keperluan' menggunakan proses yang sama seperti update status surat di fungsi sebelumnya. Metode format ID surat sama seperti format NIK penduduk.

Sementara 'nikPemohon' mengakses NIK dari 'indexUser' atau pengguna yang membuat surat tersebut. Pembuatan ID surat sama dengan pembuatan NIK, lalu simpan ke data surat dengan status 'menunggu' dan keterangan yang dikosongkan.

```
void editSuratUser(Surat dataSurat[], int jumlahSurat, int indexUser, User
dataUser[]) {
    try {
        string cariID;
        cout << "Masukkan ID surat dari surat yang ingin diubah: ";
        getline(cin, cariID);
        int cariIndex = cariSuratUser(dataSurat, jumlahSurat, cariID,
dataUser[indexUser].dataDiri.nik);
        if (cariIndex == -1) {
            throw out_of_range("Surat tidak ditemukan!");
        }
        if (!statusMenunggu(dataSurat, cariIndex)) {
            throw logic_error("Surat sudah diproses, tidak bisa diedit!");
        }
        cout << "Data ditemukan: \n";
...
string opsiUbahSurat[] = {"Jenis surat", "Keperluan surat", "Kembali"};
        int pilihan = tampilMenu("Pilih perubahan\n", opsiUbahSurat, 3);
        switch(pilihan) {
            case 1: {
                int pilihJenis = tampilMenu("Pilih jenis\n", daftarJenis,
4);
                dataSurat[cariIndex].jenisSurat = daftarJenis[pilihJenis-1];
                cout << fg::green << "Berhasil diubah!" << style::reset <<
"\n";
                break;
            }
            case 2: {
                int pilihKeperluan = tampilMenu("Pilih kepeluan",
daftarKeperluan, 4);
                dataSurat[cariIndex].keperluan =
daftarKeperluan[pilihKeperluan-1];
                cout << fg::green << "Berhasil diubah!" << style::reset <<
"\n";
                break;
            }
```

```
            case 3: {
                break;
            }
        }
    } catch (out_of_range& e) {
        cout << fg::red << style::bold << "[TIDAK DITEMUKAN]" << e.what() <<
style::reset << "\n";
    } catch (logic_error& e) {
        cout << fg::red << style::bold <<"[PERMINTAAN DITOLAK]" << e.what()
<< style::reset << "\n";
```

Source Code 3.7.3 Prosedur Pengeditan Surat Versi User

Cari terlebih dahulu surat yang ingin diubah melalui ID surat, jika surat tersebut memiliki status selain menunggu maka surat tidak dapat diedit. Perubahan hanya ada di jenis dan keperluan, proses pengeditan sama ketika menambah surat, pilih perubahan dalam daftar dan data lama akan ditimpa.

```
void hapusSuratUser(Surat dataSurat[], int &jumlahSurat, int indexUser, User
dataUser[]) {
    try {
        string cariID;
        cout << "Masukkan ID surat dari surat yang ingin diubah: ";
        getline(cin, cariID);
        int cariIndex = cariSuratUser(dataSurat, jumlahSurat, cariID,
dataUser[indexUser].dataDiri.nik);
        if (cariIndex == -1) {
            throw out_of_range("Surat tidak ditemukan!");
        }
        if (!statusMenunggu(dataSurat, cariIndex)) {
            throw logic_error("Surat sudah diproses, tidak bisa dihapus!");
        }
        cout << "Data ditemukan: \n";
...
if(!konfirmasi("Yakin ingin hapus?")) {
                cout << "Batal hapus!\n";
                return;
            }
            hapus(dataSurat, jumlahSurat, cariIndex);
            cout << fg::green << "Berhasil dihapus!" << style::reset <<
"\n";
    }  catch (out_of_range& e) {
        cout << fg::red << style::bold << "[TIDAK DITEMUKAN]" << e.what() <<
style::reset << "\n";
    } catch (logic_error& e) {
        cout << fg::red << style::bold << "[PERMINTAAN DITOLAK]" << e.what()
<< style::reset << "\n";
```

Source Code 3.7.4 Prosedur Hapus Surat Versi User

Cari surat melalui ID, jika status ‘menunggu’ maka penghapusan bisa dilakukan. Setelah konfirmasi baru surat dihapus dengan pergeseran.

3.8 Daftar Fungsi Sorting

```
void bubbleSortNamaPenduduk(Penduduk* arr, int n) {
    bool swapped;

    for (int i=0; i<n-1; i++) {
        swapped = false;

        for (int j = 0; j < n-i-1; j++) {
            if ((arr+j)->nama < (arr+j+1)->nama) {
                swap(*(arr+j), *(arr+j+1));
                swapped = true;
            }
        }
        if (!swapped) break;
```

Source Code 3.8.1 Bubble Sort

Fungsi ini mengurutkan nama penduduk secara *descending* menggunakan *bubble sort*. Proses menggunakan *pointer* dalam pengurutannya, dengan parameter ‘arr’ yang menunjuk *struct* ‘Penduduk’ dan ‘n’ yang nanti memiliki argumen jumlah penduduk.

Seperti yang dijelaskan pada bagian *flowchart*, *bubble sort* menggunakan konsep pertukaran elemen awal dengan elemen setelahnya. Dalam kasus *descending*, *loop for* i terlebih dahulu dengan batas n-1. Kita juga memiliki *flag* ‘swapped’ bernilai false.

Berikutnya *loop* j dengan nilai sama untuk membandingkan nama antara ‘arr[j] dengan ‘arr[j+1]’. Beri batas n-i-1 karena elemen di belakang sudah terurut. Jika tidak terjadi pertukaran dalam satu putaran i maka *break*.

```
void selectionSortIdSurat (Surat* arr, int n) {
    for (int i=0; i<n-1; i++) {
        int indeksMin = i;

        for (int j=i+1; j<n; j++) {
            if ((arr+j)->idSurat < (arr+indeksMin)->idSurat) {
                indeksMin = j;
            }
        }
        if(indeksMin != i) {
            swap(*(arr+i), *(arr+indeksMin));
```

Source Code 3.8.2 Selection Sort

Sorting berikutnya mengurutkan surat berdasarkan ID surat secara *ascending*. *Selection* mencari nilai terkecil terlebih dahulu yang menggunakan elemen pertama sebagai pembanding untuk mencari elemen terkecil.

Loop for i, 'indeksMin' adalah indeks elemen terkecil sejauh ini, awalan asumsi adalah i. Lalu *loop j* dimulai dari i+1, memeriksa semua elemen setelah posisi i. Mencari apakah ada ID yang lebih kecil dari 'indeksMin'. Kalau ada maka diperbarui.

Setelah satu *loop* j selesai, baris 'indeksMin != i' dijalankan, jika *true* maka *swap*. Kalau tidak maka tidak perlu *swap* karena sudah menyimpan elemen terkecil. Sesudah ini i di *increment*.

```
int prioritasStatus(string status) {
    if (status == "menunggu") return 1;
    if (status == "diproses") return 2;
    if (status == "selesai") return 3;
    if (status == "ditolak") return 4;
    return 5;
```

Source Code 3.8.3 Fungsi Prioritas Status

Fungsi ini akan digunakan pada *insertion sort*. Dalam program ini *insertion sort* mengurutkan status pada surat berdasarkan prioritas. Secara urutan menunggu->diproses->selesai->ditolak. Surat-surat akan diurutkan berdasarkan urutan tersebut. Status-status ini akan diperiksa dan mengembalikan nilai *integer*.

```
void insertionSortStatusSurat(Surat* arr, int n) {
    for (int i=1; i<n; i++) {
        Surat key = *(arr+i);
        int j = i - 1;

        while (j >= 0 && prioritasStatus((arr+j)->status) >
prioritasStatus(key.status)) {
            *(arr+j+1) = *(arr+j);
            j--;
        }
        *(arr+j+1) = key;
```

Source Code 3.8.4 Insertion Sort

Variabel i pada *loop* dimulai dari 1 karena elemen pertama sudah dianggap terurut. Variabel 'key' menyimpan elemen yang sedang ingin disisipkan. Sementara j adalah indeks elemen sebelum 'key'.

Masuk ke *loop while* dengan kondisi j masih dalam batas *array* dan elemen sebelumnya punya prioritas lebih besar. Jika kondisi terpenuhi tukar 'arr[j+1]' dan 'arr[j]'. Setelah *while* berhenti *key* juga diganti.

3.9 Main

```
Penduduk dataPenduduk[100];
    User dataUser[100];
    Surat dataSurat[100];
    int jumlahPenduduk = 7;
    int jumlahUser = 5;
    int jumlahSurat = 10;
    int totalSurat = 10;
    int totalPenduduk = 7;
    int indexUser = -1;

    dataPenduduk[0] = {"PDK004", "siti", "jl. mawar no.12", "08332"};
    dataPenduduk[1] = {"PDK007", "hendra", "jl. dahlia no.9", "08665"};
    dataPenduduk[2] = {"PDK002", "amat", "gg bersama", "08112"};
    dataPenduduk[3] = {"PDK005", "deni", "jl. melati no.3", "08443"};
    dataPenduduk[4] = {"PDK001", "radja", "gg bersatu", "08111"};
    dataPenduduk[5] = {"PDK006", "rina", "jl. kenanga no.7", "08554"};
    dataPenduduk[6] = {"PDK003", "budi", "jl. merdeka no.5", "08221"};

    dataUser[0] = {"s", "014", "user", dataPenduduk[0]};
    dataUser[1] = {"b", "013", "user", dataPenduduk[6]};
    dataUser[2] = {"a", "011", "user", dataPenduduk[2]};
    dataUser[3] = {"ri", "015", "user", dataPenduduk[5]};
    dataUser[4] = {"ra", "012", "admin", dataPenduduk[4]};

    dataSurat[0] = {"SRT005", "PDK003", "Surat Pengantar KTP", "Keperluan
pendidikan", "diproses", "Menunggu TTD lurah"};
    dataSurat[1] = {"SRT009", "PDK004", "Surat Pengantar KTP", "Melamar
pekerjaan", "diproses", "Menunggu TTD lurah"};
    dataSurat[2] = {"SRT003", "PDK003", "Surat Keterangan Domisili",
"Melamar pekerjaan", "selesai", "Sudah diverifikasi"};
    dataSurat[3] = {"SRT007", "PDK002", "Surat Keterangan Kelahiran",
"Keperluan pendidikan", "menunggu", "-"};
    dataSurat[4] = {"SRT001", "PDK002", "Surat Keterangan Tidak Mampu",
"Keperluan pendidikan", "diproses", "Dalam pengecekkan"};
    dataSurat[5] = {"SRT008", "PDK003", "Surat Keterangan Domisili",
"Keperluan pernikahan", "menunggu", "-"};
    dataSurat[6] = {"SRT010", "PDK006", "Surat Keterangan Kelahiran",
"Keperluan pendidikan", "menunggu", "-"};
    dataSurat[7] = {"SRT004", "PDK004", "Surat Keterangan Domisili",
"Melamar pekerjaan", "menunggu", "-"};
    dataSurat[8] = {"SRT006", "PDK004", "Surat Keterangan Tidak Mampu",
"Keperluan pendidikan", "ditolak", "Data tidak sesuai"};
    dataSurat[9] = {"SRT002", "PDK002", "Surat Keterangan Tidak Mampu",
"Keperluan usaha", "menunggu", "-"};
```

Source Code 3.9.1 Inisialisasi Variabel dan Array

Sebelum masuk ke program utama, ada beberapa inisialisasi. Tiga *array* dari masing-masing kategori, penetapan jumlah dan total. Lalu variabel 'indexUser' menunjukkan indeks pengguna yang sedang *login*. Data *dummy* dari masing-masing kategori untuk mempermudah program.

```
while (true) {
        cout << "Selamat datang di Website Balai Kota!\n";
        string opsiAuth[] = {"Login", "Register", "Keluar"};
        int pilihan = tampilMenu("Silahkan login untuk lanjut\n", opsiAuth,
3);
        switch(pilihan) {
```

Source Code 3.9.2 Menu Autentikasi

Di dalam *loop while* utama pengguna harus *login* untuk lanjut atau registrasi.

```
if (dataUser[indexUser].role == "admin") {
                    while (true) {
                        string opsiAdmin[] = {"Data penduduk", "Data akun",
"Data surat", "Log-out"};
                        int pilihanAdmin = tampilMenu("Menu Admin",
opsiAdmin, 4);
                        switch(pilihanAdmin) {
```

Source Code 3.9.3 Menu Admin

Jika saat *login* kesempatan pengguna habis maka program langsung berhenti dengan 'return 0'. Jika pengguna adalah admin ia bisa memilih menu yang diinginkan, penduduk, akun, atau surat.

```
case 1: {
                                while (true) {
                                    string opsiPenduduk[] = {"Lihat",
"Tambah", "Edit", "Hapus",
                                        "Urut Nama Penduduk (Desc)", "Cari
Penduduk dari Nama", "Kembali"};
                                    int pilihanPenduduk = tampilMenu("Menu
Penduduk", opsiPenduduk, 7);
                                    cin.ignore();
                                    switch(pilihanPenduduk) {
                                        case 1: tampilPenduduk(dataPenduduk,
jumlahPenduduk); break;
                                        case 2: tambahPenduduk(dataPenduduk,
jumlahPenduduk, totalPenduduk); break;
                                        case 3: editPenduduk(dataPenduduk,
jumlahPenduduk, dataUser, jumlahUser); break;
                                        case 4:
```

```
hapusPendudukAdmin(dataPenduduk, jumlahPenduduk, dataUser, jumlahUser,
dataSurat, jumlahSurat, indexUser); break;
                                        case 5: {
                                            cout << "Daftar penduduk
berdasarkan urutan nama (descending): " << "\n";
                                            Penduduk temp[100];
                                            for (int i=0; i<jumlahPenduduk;
i++) {
                                                temp[i] = dataPenduduk[i];
                                            }

bubbleSortNamaPenduduk(temp,jumlahPenduduk);
                                            tampilPenduduk(temp,
jumlahPenduduk);
                                            break;
                                        }
                                        case 6: {
                                            Penduduk temp[100];
                                            for (int i=0; i<jumlahPenduduk;
i++) {
                                                temp[i] = dataPenduduk[i];
                                            }
                                            bubbleSortNamaPenduduk(temp,
jumlahPenduduk);
                                            string cariNama;
                                            cout << "Masukkan nama penduduk:
";
                                            getline(cin, cariNama);
                                            int cariIndex = jumpSearch(temp,
jumlahPenduduk, cariNama);

                                            if (cariIndex != -1) {
                                                cout << "Ditemukan!\n";
                                                Table table;
                                                table.add_row({"NIK",
"Nama", "Alamat", "No Telp"});
                                                table.add_row({
                                                temp[cariIndex].nik,
                                                temp[cariIndex].nama,
                                                temp[cariIndex].alamat,
                                                temp[cariIndex].noTelp
                                                });

table.column(0).format().width(6); // NIK

table.column(1).format().width(16); // Nama

table.column(2).format().width(16); // Alamat
```

```
table.column(3).format().width(13); // No Telp
                                                cout << table << "\n";
                                            } else {
                                                cout << "Tidak
ditemukan!\n";
                                            }
                                            break;
                                        }
                                        case 7: break;
                                    }
                                    if (pilihanPenduduk == 7) break;
                                }
                                break;
                            }
                            case 2: {
                                while (true) {
                                    string opsiAkun [] = {"Lihat", "Hapus",
"Kembali"};
                                    int pilihanAkun = tampilMenu("Menu
Akun", opsiAkun, 3);
                                    cin.ignore();
                                    switch(pilihanAkun) {
                                        case 1: tampilAkun(dataUser,
jumlahUser); break;
                                        case 2: hapusAkun(dataUser,
jumlahUser, indexUser); break;
                                        case 3: break;
                                    }
                                    if (pilihanAkun == 3) break;
                                }
                                break;
                            }
                            case 3: {
                                while (true) {
                                    string opsiSurat[] = {"Lihat", "Update",
"Hapus", "Urut ID Surat (Asc)",
                                        "Urut Status Surat (Asc)", "Cari
Surat","Kembali"};
                                    int pilihanSurat = tampilMenu("Menu
Surat", opsiSurat, 7);
                                    cin.ignore();
                                    switch(pilihanSurat) {
                                        case 1: tampilSurat(dataSurat,
jumlahSurat); break;
                                        case 2: updateSuratAdmin(dataSurat,
jumlahSurat); break;
                                        case 3: hapusSuratAdmin(dataSurat,
```

```
jumlahSurat); break;
                                        case 4: {
                                            cout << "Daftar surat
berdasarkan ID surat (ascending)" << "\n";
                                            Surat temp[100];
                                            for (int i=0; i<jumlahSurat;
i++) {
                                                temp[i] = dataSurat[i];
                                            }
                                            selectionSortIdSurat(temp,
jumlahSurat);
                                            tampilSurat(temp, jumlahSurat);
                                            break;
                                        }
                                        case 5: {
                                            cout << "Daftar surat
berdasarkan prioritas status: " << "\n"
                                                <<
"menunggu->diproses->selesai->ditolak" << "\n";
                                            Surat temp[100];
                                            for (int i=0; i<jumlahSurat;
i++) {
                                                temp[i] = dataSurat[i];
                                            }
                                            insertionSortStatusSurat(temp,
jumlahSurat);
                                            tampilSurat(temp, jumlahSurat);
                                            break;
                                        }
                                        case 6: {
                                            string cariID;
                                            cout << "Masukkan ID surat dari
surat yang ingin dicari:";
                                            getline(cin, cariID);
                                            int cariIndex =
linearSearch(dataSurat, jumlahSurat, cariID, 3);
                                            if (cariIndex == -1) {
                                                cout << "Surat tidak
ditemukan!\n";
                                                break;
                                            }
                                            cout << "Data ditemukan: \n";
                                            Table table;
                                            table.add_row({"ID Surat", "NIK
Pemohon", "Jenis Surat",
                                                "Keperluan", "Status",
"Keterangan"});
                                            table.add_row({
```

```
dataSurat[cariIndex].idSurat,

dataSurat[cariIndex].nikPemohon,

dataSurat[cariIndex].jenisSurat,

dataSurat[cariIndex].keperluan,
                                                    dataSurat[cariIndex].status,

dataSurat[cariIndex].keterangan
                                                });

table.column(0).format().width(6); // ID Surat

table.column(1).format().width(6); // NIK Pemohon

table.column(2).format().width(17); // Jenis Surat

table.column(3).format().width(11); // Keperluan

table.column(4).format().width(8); // Status

table.column(5).format().width(20); // Keterangan
                                                cout << table << "\n";
                                                break;
                                            }
                                            case 7: break;
                                        }
                                        if (pilihanSurat == 7) break;
                                    }
                                    break;
                                }
                                case 4: {
                                    cout << fg::green << "Log-out berhasil!" <<
style::reset << "\n";
                                    indexUser = -1;
                                    break;
                                }
                            }
                            if (pilihanAdmin == 4) break;
```

Source Code 3.9.4 Proses di Setiap Menu Admin

Di menu penduduk admin dapat melihat daftar, mengedit, menghapus data penduduk, mengurutkan data penduduk berdasarkan nama, dan cari penduduk dari nama. Di menu akun dapat melihat daftar akun dan menghapus data akun. Di menu surat admin dapat melihat daftar surat, update, menghapus surat, mengurutkan surat berdasarkan ID surat dan prioritas

status serta mencari surat dari ID surat. Memilih pilihan keempat aitu *log-out* akan mengembalikan pengguna ke menu *auth* dan 'indexUser' kembali ke -1. Semua proses CRUD akan memanggil fungsi yang sesuai yang sudah dijelaskan di subbab sebelumnya.

```
else {
                    while (true) {
                        string opsiUser[] = {"Data Sendiri", "Ganti
Password", "Surat Menyurat", "Log-out"};
                        int pilihanUser = tampilMenu("Menu User", opsiUser,
4);
                        switch(pilihanUser) {
```

Source Code 3.9.5 Menu User

Role user hanya bisa melihat data sendiri, mengganti *password*, dan CRUD surat.

```
case 1: {
                                cout << "NIK: " <<
dataUser[indexUser].dataDiri.nik << "\n";
                                cout << "Nama: " <<
dataUser[indexUser].dataDiri.nama << "\n";
                                cout << "Alamat: " <<
dataUser[indexUser].dataDiri.alamat << "\n";
                                cout << "No Telp: " <<
dataUser[indexUser].dataDiri.noTelp << "\n";
```

Source Code 3.9.6 Menampilkan Data Sendiri

Menampilkan data sendiri menggunakan cara *cout* sederhana. *Field* yang ditampilkan sebenarnya berasal dari *struct* 'Penduduk' namun diakses melalui *nested struct* dalam *struct* 'User'.

```
case 2: gantiPassword(dataUser, indexUser); break;
                            case 3: {
                                while (true) {
                                    string opsiSuratUser[] = {"Lihat Surat",
"Buat Surat", "Edit Surat", "Hapus Surat",
                                        "Urut Status Surat (Ascs)", "Cari
surat","Kembali"};
                                    int pilihanSuratUser = tampilMenu("Menu
surat", opsiSuratUser, 7);
                                    cin.ignore();
                                    switch(pilihanSuratUser) {
                                        case 1: tampilSuratByNIK(dataSurat,
jumlahSurat, dataUser[indexUser].dataDiri.nik); break;
                                        case 2: buatSurat(dataSurat,
jumlahSurat, totalSurat, indexUser, dataUser); break;
                                        case 3: editSuratUser(dataSurat,
```

```
jumlahSurat, indexUser, dataUser); break;
                                        case 4: hapusSuratUser(dataSurat,
jumlahSurat, indexUser, dataUser); break;
                                        case 5: {
                                            cout << "Daftar surat
berdasarkan prioritas status: " << "\n"
                                                <<
"menunggu->diproses->selesai->ditolak" << "\n";
                                            Surat temp[100];
                                            for (int i=0; i<jumlahSurat;
i++) {
                                                temp[i] = dataSurat[i];
                                            }
                                            insertionSortStatusSurat(temp,
jumlahSurat);
                                            tampilSuratByNIK(temp,
jumlahSurat, dataUser[indexUser].dataDiri.nik);
                                            break;
                                        }
                                        case 6: {
                                            string cariID;
                                            cout << "Masukkan ID surat yang
ingin dicari: ";
                                            getline(cin, cariID);
                                            int cariIndex =
cariSuratUser(dataSurat, jumlahSurat, cariID,
dataUser[indexUser].dataDiri.nik);
                                            if (cariIndex == -1) {
                                                cout << "Surat tidak
ditemukan!\n";
                                                break;
                                            }
                                            cout << "Data ditemukan: \n";
                                            Table table;
                                            table.add_row({"ID Surat", "NIK
Pemohon", "Jenis Surat",
                                                "Keperluan", "Status",
"Keterangan"});
                                            table.add_row({

dataSurat[cariIndex].idSurat,

dataSurat[cariIndex].nikPemohon,

dataSurat[cariIndex].jenisSurat,

dataSurat[cariIndex].keperluan,
                                                dataSurat[cariIndex].status,
```

```
dataSurat[cariIndex].keterangan
                                        });

table.column(0).format().width(6); // ID Surat

table.column(1).format().width(6); // NIK Pemohon

table.column(2).format().width(17); // Jenis Surat

table.column(3).format().width(11); // Keperluan

table.column(4).format().width(8); // Status

table.column(5).format().width(20); // Keterangan
                                        cout << table << "\n";
                                        break;
                                    }
                                    case 7: break;
                                }
                                if (pilihanSuratUser == 7) break;
```

Source Code 3.9.7 Menu User Lainnya

Menu yang lain sudah memiliki fungsinya sendiri, jadi cukup panggil prosedur dan fungsi yang sesuai. Pada pilihan ke-5 *user* juga bisa mengurutkan surat berdasarkan prioritas status pada surat menggunakan *insertion sort*. Lalu pada pilihan ke-6 *user* bisa mencari surat mereka.

```
case 2: {
                cin.ignore();
                registarsi(dataUser, jumlahUser, dataPenduduk,
jumlahPenduduk); break;
            }
            case 3: {
                return 0;
```

Source Code 3.9.8 Pilihan Regis dan Keluar Program

Panggil fungsi ‘registrasi’ dengan argumen data *user* dan penduduk serta jumlah keduanya. Pilihan 3 akan mengakhiri program.

4. Hasil Output

```
Selamat datang di Website Balai Kota!

Silahkan login untuk melanjutkan
1. Login
2. Register
3. Keluar
Pilih:
```

Gambar 4.1 Tampilan Auth

```
Pilih: 2
Masukkan username baru: am
Masukkan password baru: 013
Masukkan nama lengkap: amat
Ditemukan 2 penduduk dengan nama sama:
1. amat - PDK3
2. amat - PDK4
Pilih nomor: 1
Registrasi berhasil!
Nik kamu: PDK3
```

Gambar 4.2 Registrasi dengan Nama Sama

```
Login berhasil! Selamat datang, r!


Pilih menu field:
1. Data Penduduk
2. Data akun
3. Data surat
4. Log-out
```

Gambar 4.3 Menu Admin

```
Pilih menu penduduk:
1. Lihat daftar penduduk
2. Tambah penduduk
3. Edit data penduduk
4. Hapus penduduk
5. Kembali
```

Gambar 4.4 Menu Penduduk

```
Pilih: 1
1. NIK: PDK1, Nama: radja, Alamat: gg bersatu, Nomor Telepon: 08111
2. NIK: PDK2, Nama: amat, Alamat: gg bersama, Nomor Telepon: 08112
3. NIK: PDK3, Nama: amat, Alamat: gg batu, Nomor Telepon: 08113
4. NIK: PDK4, Nama: amat, Alamat: gg putih, Nomor Telepon: 08114
```

Gambar 4.5 Daftar Penduduk

```
Pilih: 2
Masukkan nama: Omar
Masukkan alamat: gg bersatu
Masukkan nomor telepon: 08115
Berhasil ditambahkan!
```

Gambar 4.6 Tambah Penduduk

```
Pilih: 3
Masukkan NIK dari data penduduk yang ingin diubah: PDK5
NIK: PDK5, Nama: Omar, Alamat: gg bersatu, Nomor Telepon: 08115

Pilih perubahan:
1. Ubah nama penduduk
2. Ubah alamat penduduk
3. Ubah no telepon penduduk
4. Kembali
Pilih: 2
Alamat baru: gg matahari
Berhasil diubah!
```

Gambar 4.7 Mengedit Data Penduduk

```
Pilih: 4
Masukkan NIK dari data penduduk yang ingin dihapus: PDK4
NIK: PDK4, Nama: amat, Alamat: gg putih, Nomor Telepon: 08114
Konfirmasi hapus? (y/n): y
Berhasil dihapus!
```

Gambar 4.8 Menghapus Penduduk

```
Pilih: 1
1. NIK: PDK1, Nama: radja, Alamat: gg bersatu, Nomor Telepon: 08111
2. NIK: PDK2, Nama: amat, Alamat: gg bersama, Nomor Telepon: 08112
3. NIK: PDK3, Nama: amat, Alamat: gg batu, Nomor Telepon: 08113
4. NIK: PDK5, Nama: Omar, Alamat: gg matahari, Nomor Telepon: 08115
```

Gambar 4.9 Daftar Penduduk Setelah Edit dan Hapus

```
Pilih menu akun:
1. Lihat daftar akun
2. Hapus akun
3. Kembali
```

Gambar 4.10 Tampilan Menu Akun

```
Pilih: 1
1. Username: r, Role: admin, NIK: PDK1, Nama: radja
2. Username: a, Role: user, NIK: PDK2, Nama: amat
3. Username: am, Role: user, NIK: PDK3, Nama: amat
```

Gambar 4.11 Daftar Akun

```
Pilih menu surat:
1. Lihat daftar surat
2. Update surat
3. Hapus surat
4. Kembali
Pilih: 1
1. ID surat: SRT1, NIK Pemohon: PDK2, Jenis Surat: Surat Keterangan Tidak Mampu, Keperluan:
Keperluan Pendidikan, Status: diproses, Keterangan: Dalam pengecekkan
2. ID surat: SRT2, NIK Pemohon: PDK2, Jenis Surat: Surat Keterangan Tidak Mampu, Keperluan:
Keperluan usaha, Status: menunggu, Keterangan: -
```

Gambar 4.12 Menu Surat dan Lihat Daftar Surat

```
Pilih: 2
Masukkan ID dari surat yang ingin diupdate: SRT1
ID surat: SRT1, NIK Pemohon: PDK2, Jenis Surat: Surat Keterangan Tidak Mampu, Keperluan: Kep
erluan Pendidikan, Status: selesai, Keterangan: Dalam pengecekkan

Pilih perubahan:
1. Ubah status
2. Ubah keterangan
3. Kembali
Pilih: 2
Isi keterangan surat: Sudah dikirimkan
Keterangan berhasil diubah!
Pilih menu surat:
1. Lihat daftar surat
2. Update surat
3. Hapus surat
4. Kembali
Pilih: 3
Masukkan ID dari surat yang ingin dihapus: SRT2
ID surat: SRT2, Jenis: Surat Keterangan Tidak Mampu, Status: menunggu
Konfirmasi hapus? (y/n): y
Berhasil dihapus!
```

Gambar 4.13 Edit dan Hapus Surat

```
Silahkan login untuk melanjutkan
Username: am
Password: 013

Login berhasil! Selamat datang, am!


Pilih menu:
1. Data sendiri
2. Ganti password
3. Surat menyurat
4. Log-out
```

Gambar 4.14 Menu User

```
Data diri kamu
NIK: PDK3
Nama: amat
Alamat: gg batu
Nomor Telepon: 08113
```

Gambar 4.15 Data Sendiri

```
Pilih: 2
Masukkan password sekarang untuk mengganti password:
013
Masukkan password baru: 015
Konfirmasi password baru: 015
Password berhasil diubah!
```

Gambar 4.16 Mengubah Password

```
Pilih: 2
Buat surat: Pilih jenis:
1. Surat Keterangan Domisili
2. Surat Keterangan Tidak Mampu
3. Surat Pengantar KTP
4. Surat Keterangan Kelahiran
Pilih: 3
Pilih keperluan:
1. Melamar pekerjaan
2. Keperluan pendidikan
3. Keperluan pernikahan
4. Lainnya
Pilih: 4
Surat berhasil ditambahkan!
```

Gambar 4.17 Buat Surat

```
Masukkan ID surat yang ingin diubah:
SRT4
Pilih perubahan:
1. Jenis surat
2. Keperluan surat
3. Kembali
Pilih: 2
Pilih keperluan:
1. Melamar pekerjaan
2. Keperluan pendidikan
3. Keperluan pernikahan
4. Lainnya
Pilih: 2
Keperluan surat berhasil diubah!
```

Gambar 4.18 Edit Surat User

```
Daftar surat:
2. ID: SRT4, Jenis: Surat Pengantar KTP, Keperluan: Keperluan pendidikan, Status: menunggu,
Keterangan: -
```

Gambar 4.19 Daftar Surat User

```
Selamat datang di Website Balai Kota!
+-----------------------------+
| Silahkan login untuk lanjut |
+-----------------------------+
| 1. Login                    |
+-----------------------------+
| 2. Register                 |
+-----------------------------+
| 3. Keluar                   |
+-----------------------------+
Pilih:
```

Gambar 4.20 Tampilan dengan Tabel di Menu Awal

```
+----+------+------+----------------+-----------+--------+-------------------+
| No | ID   | NIK  | Jenis Surat    | Keperluan | Status | Keterangan        |
|    | Sur- | Pem- |                |           |        |                   |
|    | at   | ohon |                |           |        |                   |
+----+------+------+----------------+-----------+--------+-------------------+
| 1  | SRT1 | PDK2 | Surat          | Keperluan | dipro- | Dalam pengecekkan |
|    |      |      | Keterangan     | Pendidik- | ses    |                   |
|    |      |      | Tidak Mampu    | an        |        |                   |
+----+------+------+----------------+-----------+--------+-------------------+
| 2  | SRT2 | PDK2 | Surat          | Keperluan | menun- | -                 |
|    |      |      | Keterangan     | usaha     | ggu    |                   |
|    |      |      | Tidak Mampu    |           |        |                   |
+----+------+------+----------------+-----------+--------+-------------------+
```

Gambar 4.21 Tampilan Daftar Surat dengan Tabel

```
Pilih: 5
Daftar penduduk berdasarkan urutan nama (descending):
+----+------+----------------+----------------+-------------+
| No | NIK  | Nama           | Alamat         | No Telp     |
+----+------+----------------+----------------+-------------+
| 1  | PDK- | siti           | jl. mawar      | 08332       |
|    | 004  |                | no.12          |             |
+----+------+----------------+----------------+-------------+
| 2  | PDK- | rina           | jl. kenanga    | 08554       |
|    | 006  |                | no.7           |             |
+----+------+----------------+----------------+-------------+
| 3  | PDK- | radja          | gg bersatu     | 08111       |
|    | 001  |                |                |             |
+----+------+----------------+----------------+-------------+
| 4  | PDK- | hendra         | jl. dahlia     | 08665       |
|    | 007  |                | no.9           |             |
+----+------+----------------+----------------+-------------+
| 5  | PDK- | deni           | jl. melati     | 08443       |
|    | 005  |                | no.3           |             |
+----+------+----------------+----------------+-------------+
| 6  | PDK- | budi           | jl. merdeka    | 08221       |
|    | 003  |                | no.5           |             |
+----+------+----------------+----------------+-------------+
| 7  | PDK- | amat           | gg bersama     | 08112       |
|    | 002  |                |                |             |
+----+------+----------------+----------------+-------------+
```

Gambar 4.22 Tabel Urutan Nama Penduduk dengan Bubble Sort

1	SRT-001	PDK-002	Surat Keterangan Tidak Mampu	Keperluan pendidik-an	dipro-ses	Dalam pengecekkan
2	SRT-002	PDK-002	Surat Keterangan Tidak Mampu	Keperluan usaha	menun-ggu	-
3	SRT-003	PDK-003	Surat Keterangan Domisili	Melamar pekerjaan	seles-ai	Sudah diverifikasi
4	SRT-004	PDK-004	Surat Keterangan Domisili	Melamar pekerjaan	menun-ggu	-
5	SRT-005	PDK-003	Surat Pengantar KTP	Keperluan pendidik-an	dipro-ses	Menunggu TTD lurah
6	SRT-006	PDK-004	Surat Keterangan Tidak Mampu	Keperluan pendidik-an	ditol-ak	Data tidak sesuai
7	SRT-007	PDK-002	Surat Keterangan Kelahiran	Keperluan pendidik-an	menun-ggu	-
8	SRT-008	PDK-003	Surat Keterangan Domisili	Keperluan pernikah-an	menun-ggu	-
9	SRT-009	PDK-004	Surat Pengantar KTP	Melamar pekerjaan	dipro-ses	Menunggu TTD lurah
10	SRT-010	PDK-006	Surat Keterangan Kelahiran	Keperluan pendidik-an	menun-ggu	-

Gambar 4.23 Tabel Urutan Surat berdasarkan ID Surat

No	ID Surat	NIK Pemohon	Jenis Surat	Keperluan	Status	Keterangan
1	SRT-007	PDK-002	Surat Keterangan Kelahiran	Keperluan pendidikan	menunggu	-
2	SRT-008	PDK-003	Surat Keterangan Domisili	Keperluan pernikahan	menunggu	-
3	SRT-010	PDK-006	Surat Keterangan Kelahiran	Keperluan pendidikan	menunggu	-
4	SRT-004	PDK-004	Surat Keterangan Domisili	Melamar pekerjaan	menunggu	-
5	SRT-002	PDK-002	Surat Keterangan Tidak Mampu	Keperluan usaha	menunggu	-
6	SRT-005	PDK-003	Surat Pengantar KTP	Keperluan pendidikan	diproses	Menunggu TTD lurah
7	SRT-009	PDK-004	Surat Pengantar KTP	Melamar pekerjaan	diproses	Menunggu TTD lurah
8	SRT-001	PDK-002	Surat Keterangan Tidak Mampu	Keperluan pendidikan	diproses	Dalam pengecekkan
9	SRT-003	PDK-003	Surat Keterangan Domisili	Melamar pekerjaan	selesai	Sudah diverifikasi
10	SRT-006	PDK-004	Surat Keterangan	Keperluan pendidik-	ditolak	Data tidak sesuai

Gambar 4.24 Tabel Urutan Surat berdasarkan Prioritas Status

```
Daftar surat berdasarkan prioritas status:
menunggu->diproses->selesai->ditolak
+----+------+-----------------+----------+--------+-------------------+
| No | ID   | Jenis Surat     | Keperluan | Status | Keterangan        |
|    | Sur- |                 |          |        |                   |
|    | at   |                 |          |        |                   |
+----+------+-----------------+----------+--------+-------------------+
| 1  | SRT- | Surat           | Keperluan | menun- | -                 |
|    | 007  | Keterangan      | pendidik- | ggu    |                   |
|    |      | Kelahiran       | an       |        |                   |
+----+------+-----------------+----------+--------+-------------------+
| 2  | SRT- | Surat           | Keperluan | menun- | -                 |
|    | 002  | Keterangan      | usaha    | ggu    |                   |
|    |      | Tidak Mampu     |          |        |                   |
+----+------+-----------------+----------+--------+-------------------+
| 3  | SRT- | Surat           | Keperluan | dipro- | Dalam pengecekkan |
|    | 001  | Keterangan      | pendidik- | ses    |                   |
|    |      | Tidak Mampu     | an       |        |                   |
+----+------+-----------------+----------+--------+-------------------+
```

Gambar 4.25 Tabel Urutan Prioritas Status User

```
Pilih: 3
Masukkan NIK dari data penduduk yang ingin diubah: PDK003
Data ditemukan:
+------+----------------+----------------+-------------+
| NIK  | Nama           | Alamat         | No Telp     |
+------+----------------+----------------+-------------+
| PDK- | budi           | jl. merdeka    | 08221       |
| 003  |                | no.5           |             |
+------+----------------+----------------+-------------+
```

Gambar 4.26 Mencari Penduduk dari NIK

```
Masukkan nama penduduk: amat
Ditemukan!
+------+----------------+----------------+-------------+
| NIK  | Nama           | Alamat         | No Telp     |
+------+----------------+----------------+-------------+
| PDK- | amat           | gg bersama     | 08112       |
| 002  |                |                |             |
+------+----------------+----------------+-------------+
```

Gambar 4.27 Mencari Penduduk dari Nama

```
Pilih: 1
Username: ra
Password: 012
Login berhasil! Selamat datang, ra!
```

Gambar 4.28 Output Berwarna

5. Langkah-langkah GIT

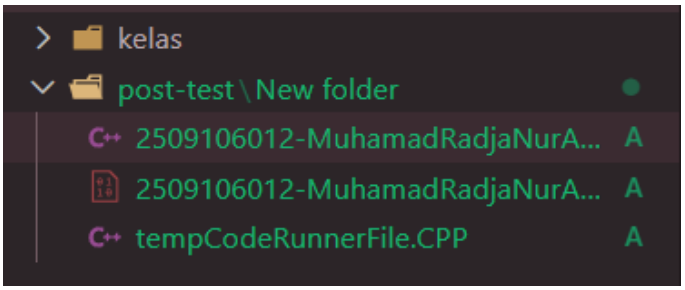
Sebelum melakukan proses *Git*, kita harus memiliki akun GitHub dan membuat *repository* baru. Lalu siapkan *folder* yang akan di git dan buka Command Prompt, PowerShell, atau semacamnya untuk melakukan *git*. Karena sudah melakukan *git init* dan *git remote*, maka prosesnya tidak perlu ditampilkan lagi.

5.1 Git Add

```
PS D:\praktikum-apl> git add .
```

Gambar 5.1.1 Proses Git Add

Git add berfungsi untuk memindah semua perubahan di area kerja ke indeks, ada dua cara untuk melakukan *git add*, yang pertama dengan mengetikkan nama *file* yang ingin di add. Jika kalian ingin menambahkan banyak *file* sekaligus, ketik "." Setelah *add* maka semua *file* yang di dalam area kerja akan ditambahkan.



Gambar 5.1.2 Setelah Git Add

5.2 Git Commit

```
PS D:\praktikum-apl> git commit -m "finish pt 1"
[master (root-commit) e8f9088] finish pt 1
 3 files changed, 98 insertions(+)
 create mode 100644 post-test/New folder/2509106012-MuhamadRadjaNurAkbar-PT-1.CPP
 create mode 100644 post-test/New folder/2509106012-MuhamadRadjaNurAkbar-PT-1.exe
 create mode 100644 post-test/New folder/tempCodeRunnerFile.CPP
```

Gambar 5.2.1 Proses Commit

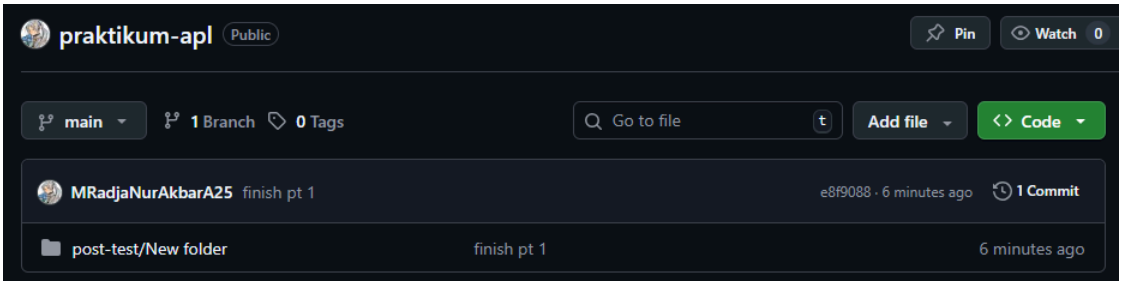
Git commit berfungsi untuk mengkonfirmasi setiap perubahan pada *repository* kalian dengan mengetikkan "git commit -m 'pesan yang ingin ditulis' " .

5.3 Git Push

```
PS D:\praktikum-apl> git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 667.93 KiB | 7.95 MiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
```

Gambar 5.3.1 Git Push

Git push berfungsi untuk mengunggah *file* lokal tadi ke GitHub kalian dengan mengetikkan "git push -u origin main. Kembali ke *repository* di GitHub kalian, klik *refresh* dan *file* sudah terunggah.



Gambar 5.3.2 File Terunggah